

Introduction to DÆDÆLUS

Please ask
questions about
anything you don't
understand

Welcome David V1.0 (02025-Jul-08)

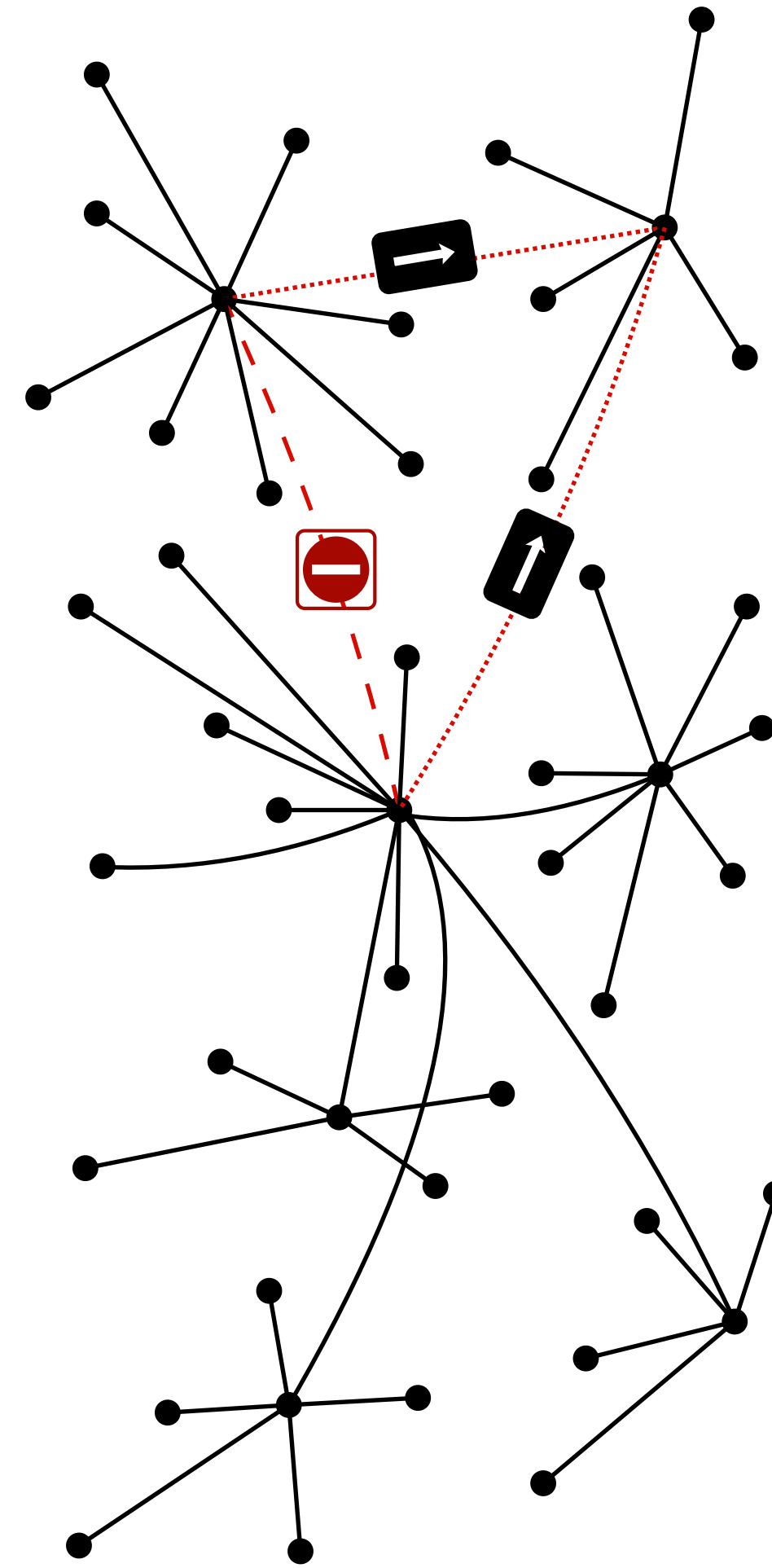
Problem

Maintaining liveness and synchronizing processes in networks is a challenge when packets can be dropped, reordered, duplicated or delayed

- Conventional networks require protocol stacks and applications to use timeouts and retries to maintain liveness
- This makes exactly-once semantics *impossible* and precipitates retry storms which lead to unbounded tail latency and transaction failure
- This results in silent corruption of data structures and loss of data.
A problem seen in every distributed database for decades
- These failures are not understood because customers (under NDA) may not publish results that embarrass vendors

Network Faults Lead to Transaction Failure

- 80% of failures have a **catastrophic impact**, with data loss being the most common (27%)
- 90% of the **failures are silent**, the rest produce warnings that are unclear
- 21% of the failures lead to permanent damage to the system. This **damage persists** even after the network partition heals



Databases Fail with Existing Networks

Partial Network Partitioning

BASIL ALKHATIB, University of Waterloo, Canada
SREEHARSHA UDAYASHANKAR, University of Waterloo, Canada
SARA QUNAIBI, University of Waterloo, Canada
AHMED ALQURAAN, University of Waterloo, Canada
MOHAMMED ALFATAFTA, University of Waterloo, Canada
Wael AL-MANASRAH, University of Waterloo, Canada
ALEX DEPOUTOVITCH, Huawei Research Canada, Canada
SAMER AL-KISWANY, University of Waterloo, Canada

We present an extensive study focused on partial network partitioning. Partial network partitions disrupt the communication between some but not all nodes in a cluster. First, we conduct a comprehensive study of system failures caused by this fault in 13 popular systems. Our study reveals that the studied failures are catastrophic (e.g., lead to data loss), easily manifest, and are mainly due to design flaws. Our analysis identifies vulnerabilities in core systems mechanisms including scheduling, membership management, and ZooKeeper-based configuration management.

Second, we dissect the design of nine popular systems and identify four principled approaches for tolerating partial partitions. Unfortunately, our analysis shows that implemented fault tolerance techniques are inadequate for modern systems; they either patch a particular mechanism or lead to a complete cluster shutdown, even when alternative network paths exist.

Finally, our findings motivate us to build Nifty, a transparent communication layer that masks partial network partitions. Nifty builds an overlay between nodes to detour packets around partial partitions. Nifty provides an approach for applications to optimize their operation during a partial partition. We demonstrate the benefit of this approach through integrating Nifty with VoltDB, HDFS, and Kafka.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; **Reliability**; **Availability**; • **Networks** → **Network reliability**.

Additional Key Words and Phrases: network failures, fault tolerance, partial network partitions, distributed systems, reliability.

1 INTRODUCTION

Modern networks are complex. They use heterogeneous hardware and software [1], deploy diverse middleboxes (e.g., NAT, load balancers, and firewalls) [2–4], and span multiple data centers [2, 4]. Despite the high redundancy built into modern networks, catastrophic failures are common [1, 3, 5, 6]. Nevertheless, modern cloud systems

Authors' addresses: Basil Alkhatib, b2alkhatib@uwaterloo.ca, University of Waterloo, Canada; Sreeharsha Udayashankar, sreeharsha.udayashankar@uwaterloo.ca, University of Waterloo, Canada; Sara Qunaibi, squnaibi@uwaterloo.ca, University of Waterloo, Canada; Ahmed Alquraan, ahmed.alquraan@uwaterloo.ca, University of Waterloo, Canada; Mohammed Alfatafta, m.alfatafta@uwaterloo.ca, University of Waterloo, Canada; Wael Al-Manasrah, wael.al-manasrah@uwaterloo.ca, University of Waterloo, Waterloo, Canada; Alex Depoutovitch, alex.depoutovitch@huawei.com, Huawei Research Canada, Canada; Samer Al-Kiswany, University of Waterloo, Canada, salkiswany@uwaterloo.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2022 Association for Computing Machinery.

0734-2071/2022/12-ART \$15.00

<https://doi.org/10.1145/3576192>

PNP

DEDELUS

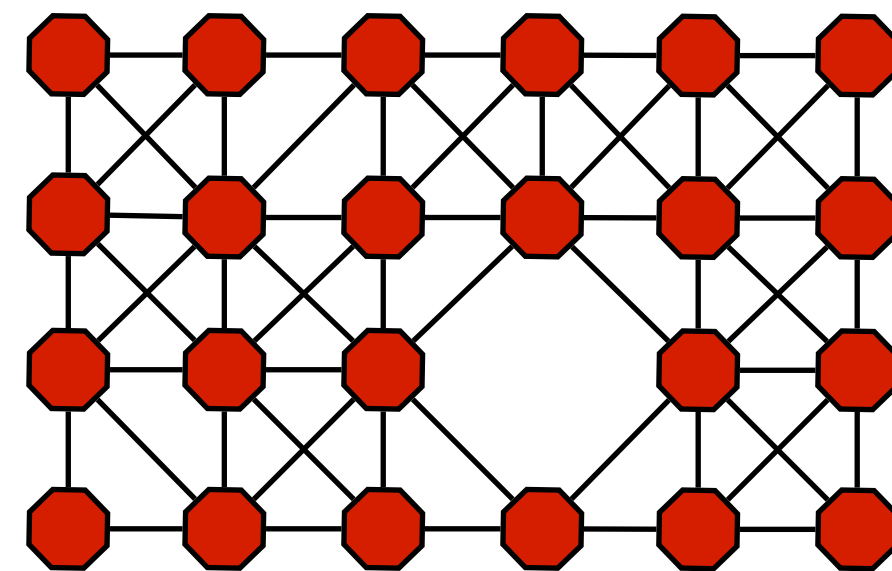
ACM Trans. Comput. Syst.



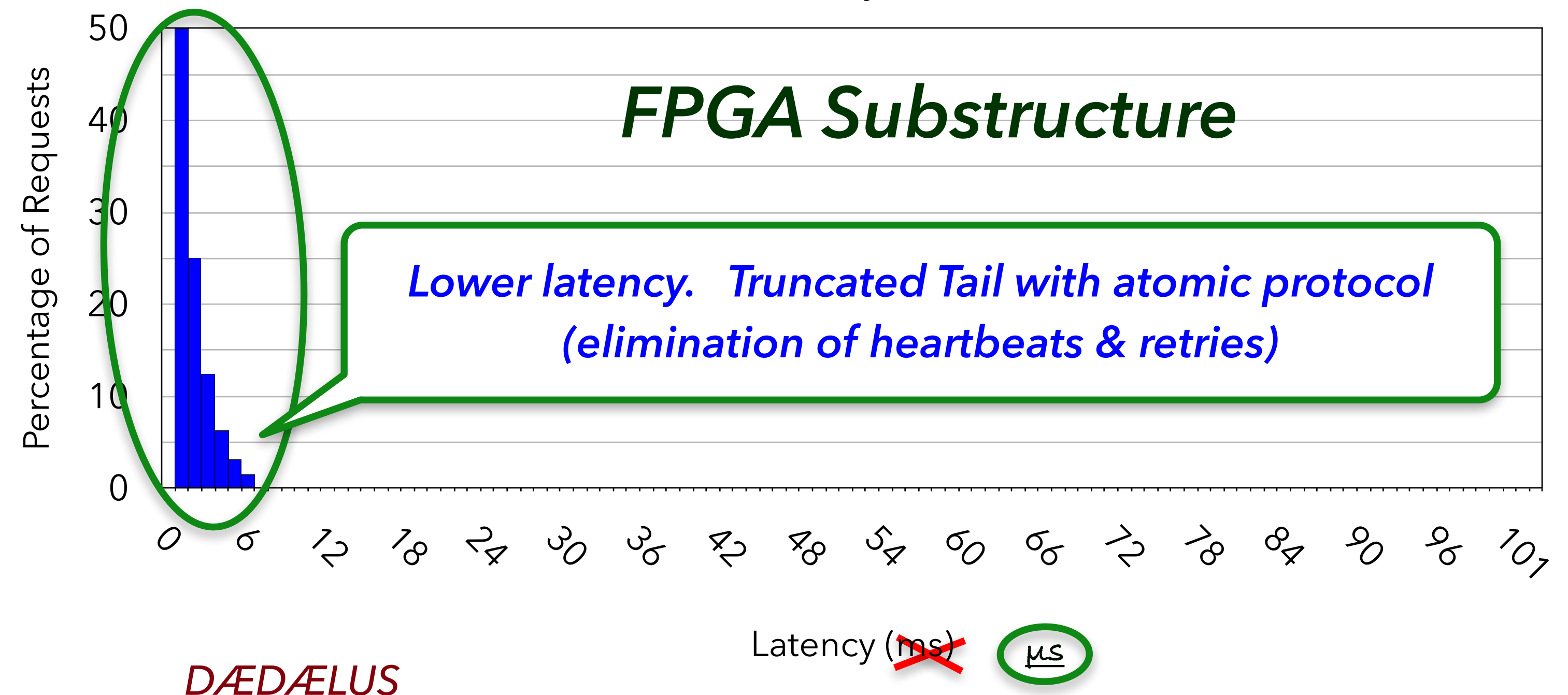
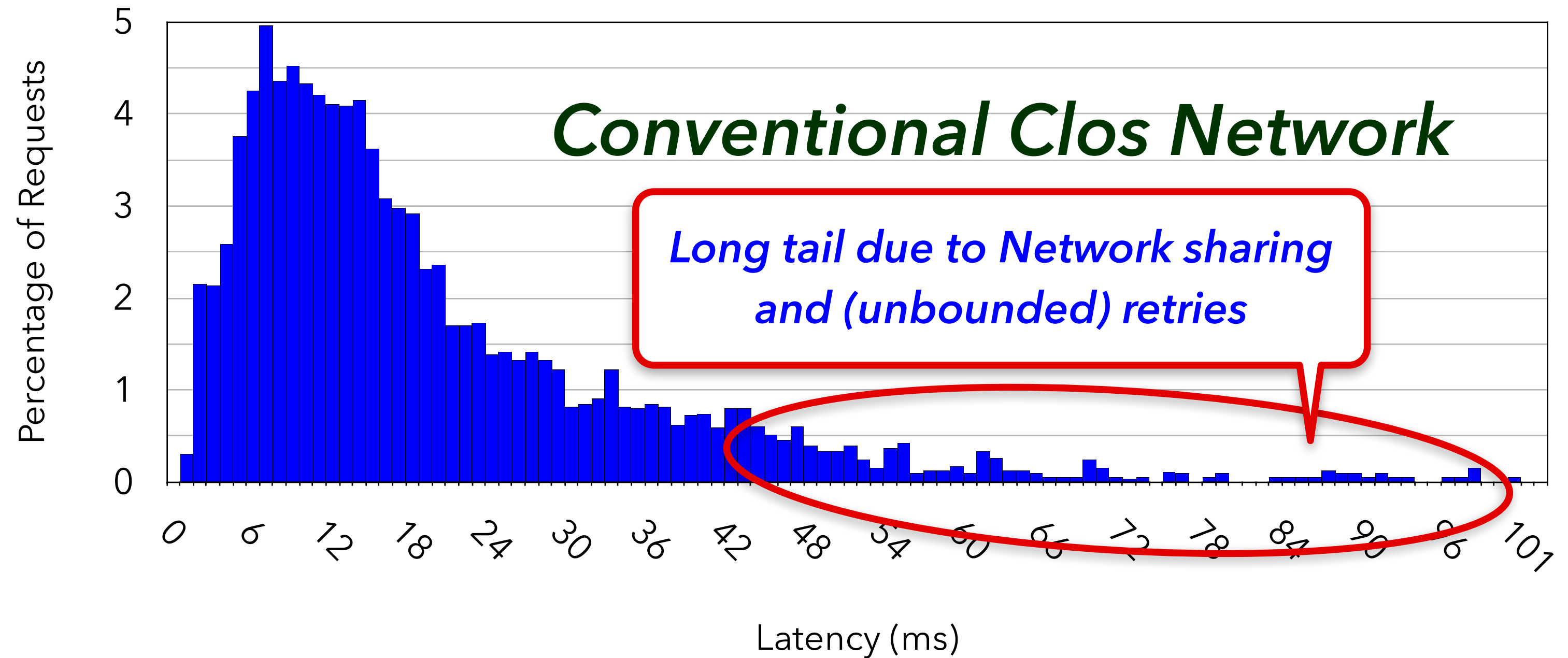
Tail Latency

Daedaelus reduces latency in ways conventional networks cannot:

- ▶ Direct connections
- ▶ Multicast consensus, in parallel over 8 ports instead of serial over 1
- ▶ Truncated Tail Latency – protocol knows it failed or succeeded (without heartbeats or timeouts)



Daedaelus



Mission

Daedaelus: Architecting the Future with Enduring Excellence

We address fundamental problems in distributed systems using protocols, data structures and algorithms inspired by Quantum Information Theory and Multiway Systems.

Our market is secure, reliable, distributed computing on the edge. Use-cases include Transaction Processing, Digital Twins, AI/ML, and Interface to Quantum Computers

- **Applied Graph Theory**
 - Digital Twins
 - Ethernet & Other Chiplet Interconnects ([UCIe](#))
- **Distributed infrastructures where:**
 - ‘Time-reversible’ constructors (specified as Petri nets) no longer rely on the irreversible smash and restart of Shannon information to recover from failures, or to reconfigure computational resources.
 - ‘Precise Information-theoretic’ emulators to enable competitive interactions for ‘Digital Twins’.
 - ‘Rulesets’ to enable non-zero-sum game (NZSG) outcomes, where winners are those who exhibit cooperative (trustable) behavior, and cheaters cannot hide behind a veil of anonymity or misinformation.

Read Me First

- *Don't let anything go by you don't understand at this stage. Ask questions!*

Rethinking Datacenter Fabrix

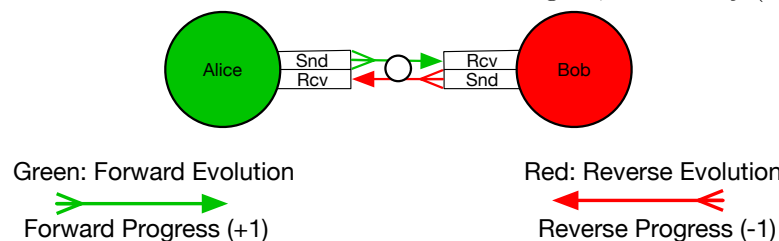
Many problems encountered in datacenters today arise from our inability to distinguish between a node that is merely slow from a node that has failed or become unreachable due to a network failure.

We take the two most recognizable elements in datacenters today: *servers* and *switches*, and refactor them into simpler, more foundational elements (fully *independent* failure domains): **cells** and **links**. A **cell** is a single type of node element (autonomous unit of compute, storage and packet processing). A **link** is an individual, bidirectional, computation object (an autonomous communication entity between *two cells*)¹.

A consequence of the former is that unifying node elements makes things simpler because we have only one type of node to manage instead of two. The consequence of the latter is profoundly more interesting: we raise the notion of a **link** to *first order* – a first-class citizen in the infrastructure – a bipartite² element of information with two *complementary* halves – *persistable* through failure and recovery events. i.e., a communication object that doesn't rule out that some fault-detection and computation is involved.

An example link utility is *The I Know That You Know That I Know* (TIKTYKTIK) property; which enables us to address some of the most difficult and pernicious problems in *distributed systems* today.

Another example link utility is *Atomic Information Transfer* (AIT). Unlike *replicated* state machines³ used throughout distributed applications today, **links** are *single* state machines: the two halves of which maintain *temporal intimacy* through hidden packet exchanges. When a local agent or actor is ready, the AIT protocol transfers *indivisible* tokens across the link to the other agent, *atomically* (all or nothing)⁴.



These TIKTYKTIK and AIT properties are *composable*. Trees of links provide a resilient *conserved quantities* mechanism to reliably distribute tokens among *agents* on an application graph. Intermediate cells *promise*⁵ to never lose AIT tokens. This defends against lost tokens because if any part of the chain (or tree) breaks, alternate paths are available to seamlessly recover the conserved quantity and continue operation.

By strengthening the system model, **links** and AIT provide a general foundation to solve many distributed systems problems⁶, such as failure-detection, consensus and distributed transactions.

Failure Modes

One might imagine we could achieve the properties of **links** over existing switched networks. If each host (or its NIC) maintains its half of the *shared state*, then shouldn't the switched network be able to act as a proxy for a single *logical* link? When a switched network fails, and reroutes, can't the two sides (NICs) just stitch the two halves of the *shared state* back together again?⁷

This simple hazard analysis⁸ misses a fundamental issue: *networks don't maintain state on behalf of applications*. Switches drop packets (and state) whenever *they* feel like it, so there are many more ways for *logical links* to get *confused over switched networks* and compromise the integrity of the *shared state*.

Key issue: Switched networks may drop packets anywhere along the path; eradicating state *and* events needed to maintain *promises* and *liveness* respectively. When a **link** fails, both sides are preserved. If there is a failure in the *token transfer* it can always be detected, and retransmissions occur only on a *real* failure (such as disconnection—where alternative routes are explicitly coordinated with applications), thus enforcing that tokens have no duplicate or out of order deliveries on the **link**^{9,10}.

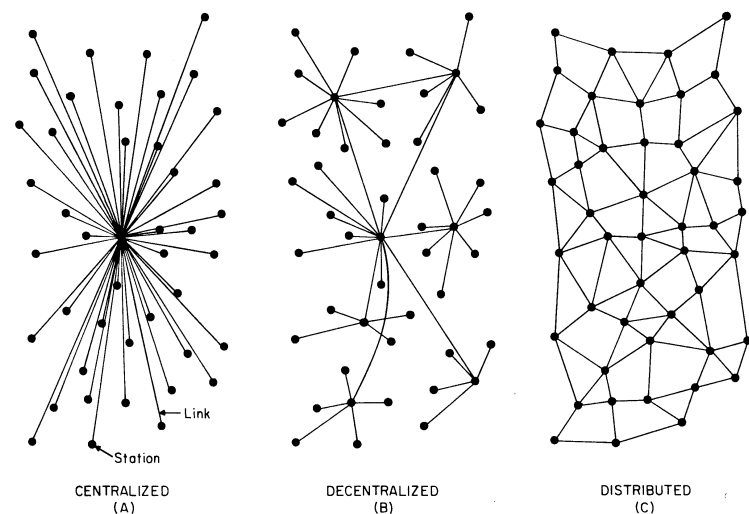
Rethinking Datacenter Management

Owners and operators of the network determine the relationships among distributed applications today. Minimum spanning trees, on which all routing is done, are built, and torn down, by *switches*; based on protocols standardized long ago when we first learned how our computers could communicate.

Today's datacenter architects build their infrastructures using two kinds of boxes: *switches*¹ and *servers*². They connect them using individual cables, which they bundle together to make them convenient to route within and around physical structures. This forms a *centralized* or *decentralized*³ topology, where the switches become hubs and servers become leaves.

Paul Baran's [classification](#) provides insight:

CENTRALIZED (A) shows 46 univalent nodes connected to a special high radix or *valency*⁴ hub. If the central hub dies, all nodes are cut off. DECENTRALIZED (B) shows 47 nodes and links, 7 nodes with a valency of 5-7 serve as switches, and 40 as univalent (leaf) nodes. If one of the switches fails, the network fractures into isolated partitions; and only nodes within the partitions can continue to communicate locally. DISTRIBUTED (C) shows 47 identical, multivalent (valency ~ 5) nodes, and 98 links. The network has better resilience: failed nodes are routed around, and many links must fail before *any* node is finally isolated.



Datacenter Topologies

CENTRALIZED topologies are avoided because they represent bottlenecks and have a single point of failure. DECENTRALIZED topologies⁵ are most prevalent, which is surprising given how *non-optimal* they are from a perspective of distributed microservices⁶, container life-cycles⁷, migration, elasticity and resilience^{8,9}. Switches that perpetuate this model, are embarrassingly complex, unreliable, arcane, and parochial. This results in very high operational costs, poor security/high vulnerability, and nothing close to five nines reliability¹⁰. DISTRIBUTED topologies are rarely used (so far) in datacenters, except for a few HPC applications¹¹.

However, within the same *or lower* capital cost, DISTRIBUTED topologies provide: greater resilience, lower latencies, higher available bandwidth and far more flexibility; by connecting **cells**¹² directly with neighbor to neighbor (N2N) connections rather than through a switched or *aggregated* network¹³. By not perpetuating the management complexity of switched networks, and introducing new, simpler, control/forwarding planes through **cells**, we can also dramatically lower operational costs.

Perhaps the time has come to recognize the genius of Paul Baran's insights, and ask why DISTRIBUTED topologies are not deployed in datacenters, where their resilience and security can be readily exploited?

Datacenter Programmability

Two types of teams co-evolved to manage modern datacenters: one to design and manage the networks, and one to program and manage the servers. This worked when datacenters had a single owner or tenant, their applications and physical infrastructure evolved slowly, and different business units could work within their own silo's. This is no longer a viable architecture in today's highly dynamic multi-tenant datacenters.

Distributed applications can no longer afford to be held back by the slow pace of networking innovation.

FAQ: GVM

Graph Virtual Machine (GVM): Naming, Topology, Equations

DAEDAELUS (DAE) has developed a low-cost, high-performance Transaction FabrixTM for datacenters using N2N (Neighbor-to-Neighbor) links (repurposing Ethernet frames), with selectable (reliable, tree cast) delivery based on a new way to rendezvous on *trees* (without fixed IP endpoint names). The lowest level graph is a set of black trees, where each black tree in the set is named by the cell on which they are rooted; together they form our *groundplane*. The architecture is layered upwards through logical stacked trees that provide hardware-enforced *confinement*, and virtual stacked trees that provide a foundation on which developers can automatically provision microservice graphs with *an equation*.

Introduction

The DAEDAELUS (DAE) Transaction Fabrix (TF) uses standard Ethernet NICs. However, East-West interactions use tree-based naming for address groups of closely related microservices in *sets*, rather than the source/destination addressing mode that practitioners of conventional networks are accustomed to¹.

This simple change, implemented *below* L2, is the first step to enabling *Structured Topology Management* of microservice sets that need to evolve dynamically in response to perturbations (failures, disasters, attacks).

With 8 ports per cell, a 1-hop cluzter comprises 9 cells. The center cell is the root of its own tree, and the default transaction manager for consensus operations. Neighbor cells become proxies for resource (compute/memory/storage) elasticity, dynamic load balancing and failover. A preselected neighbor is provided for voluntary (or involuntary) failover when the center cell dies or needs to be taken out of service.

Trees extend to any number of hops, out to the edge of the Transaction Fabrix (the boundary to the conventional network). The primary means of addressing is *radial* (by port) - it knows which 'direction' a target entity exists along, but may not know 'how far' along a branch, or which sub-branch it exists. This is an essential mechanism – to allow the migration of an entity without having to change addresses from the point of view of the source requestor. This mechanism also enables failover and elastic growing and shrinking of a microservice set in response to traffic demand.

Trees are subsetting to enable hardware confinement between zones. These are used typically to create tenants and subtenants within a datacenter. All these API's are directly available to the distributed systems developer through an API. Within the Transaction Fabrix, there is no need for conventional datacenter segmentation using firewalls, iptables, or ACL's in the routers.

The central mechanism driving this simple yet powerful approach to datacenter address management is the Graph Virtual Machine (GVM): A protected virtual machine environment running in the FPGA Substructure (SmartNICs). All packets going into a cell go through this vantage point enables: routing to be done differently, provisioning to be done differently, and security to be done differently.

Fundamental organizing unit: Graphs, not Lists

The entire datacenter is a graph (groundplane). All (non-trivial) applications in datacenters are sub-graphs. We create sub-graphs by recursively stacking trees above the groundplane. These sub-graphs are the regions or tenants that respond to commands from an orchestrator to do something. Within each subtenant, developers can orchestrate their own microservice sets without being required to work within the constraints of conventional networking tools, such as command line interfaces and ACL's in switches/routers, or the cumbersome sets of rules used in today's internal segregation firewalls.

It is inevitable that the above description will create more questions than it answers. The rest of this document is a deeper dive for the technically curious.

This FAQ is one of many in the DAEDAELUS repository.

¹This FAQ assumes that the reader has read the first two 'read-me-first' documents: [Exec-Summary-Links](#) (Rethinking Datacenter Fabrix) and [Exec-Summary-TRAPH.pdf](#) (Rethinking Datacenter Management)



Responsibilities

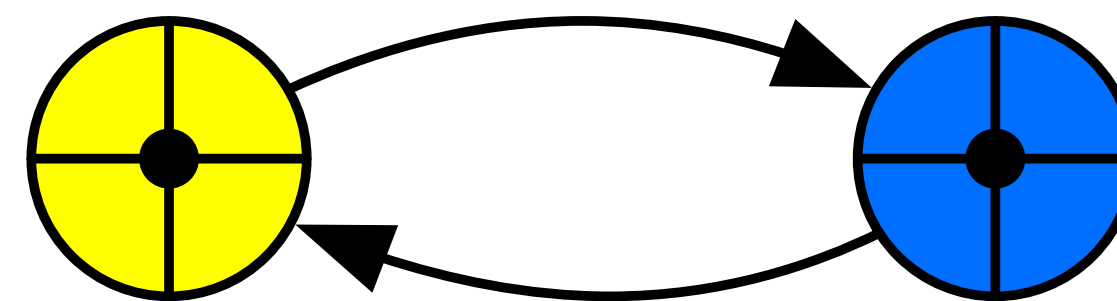
- [Paul] Vision, mission, use cases, product requirements
- [Steve] Technology Context: Networking, Compute, Storage
- [Sahas] Implementation. Emulation Rig. Bits on the Wire Demo
- [Dean] Web sites, OAE administration, Mathematica models
- [David] OAE Specifications, Tools, RTL/Verilog, Verification, IEEE
 - [?] Develop & verify API's & Petri-Net models for Labyrinth
- [Dugan] Simulation, Visualization, Artistry, UI
- [Sumanth, Varun, Gia] Testing/Review/Proof-reading, Learning

Multiway Link Protocol

Trapped (circulating) “information” token on link, exchanged with strata above, or strata below for token *conservation* in and between layers

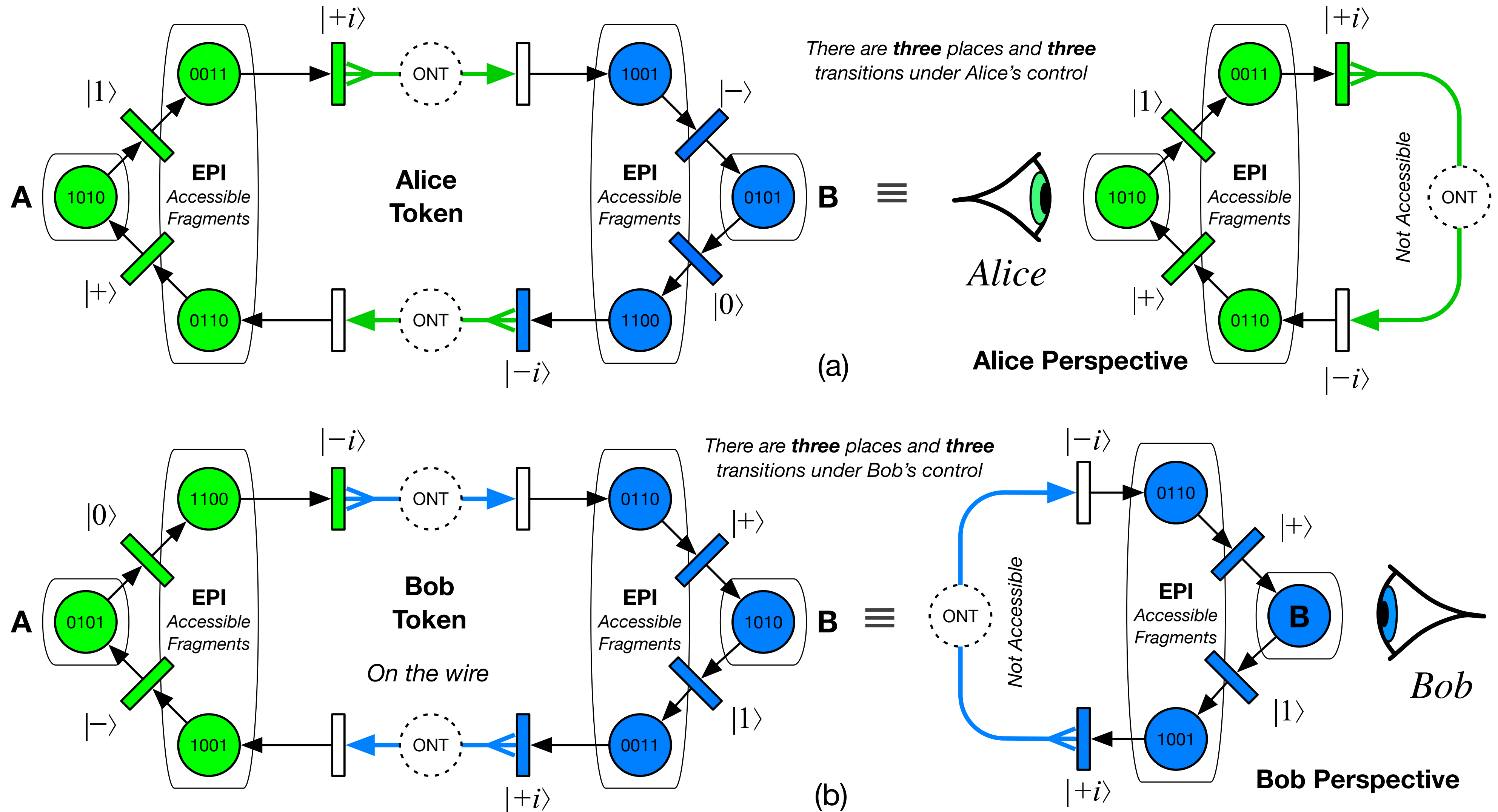
- Interfaces for logical time (Lingua Franca) reversal ancilla for [reactors](#)
- Causal invariance leads to precise definition of confined Shannon information recovery (based on knowledge from Cluster Liveness), in the face of *all hazards* presented via fault injection, via externally accessible ancilla from the *next* level up. i.e., the ONT Petri place and oracle

Superposition of Arrow Head & Tail.
Indicates Alternating causality is
occurring under the hood, but you can't see
It because reads are destructive.

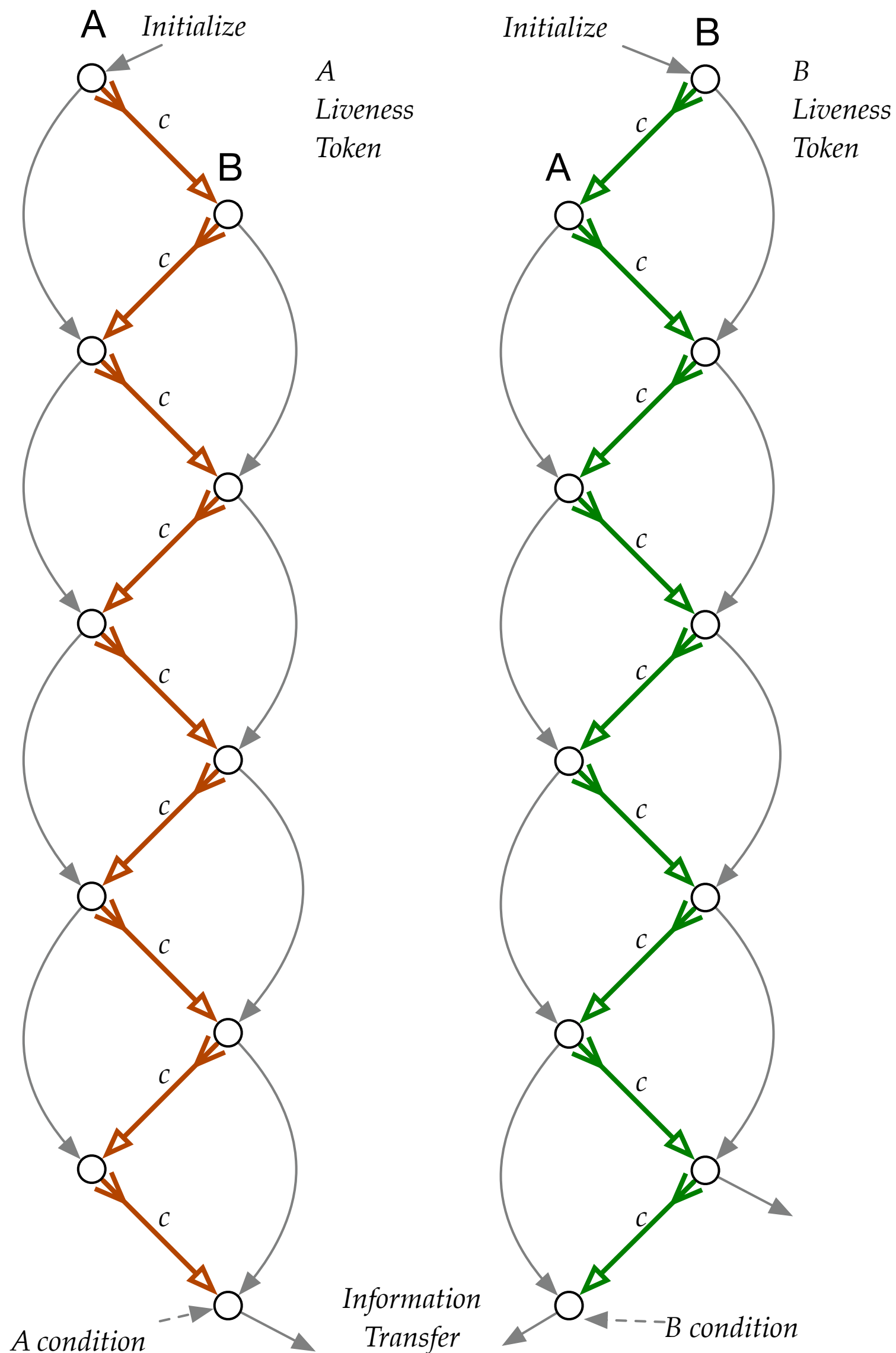


We separate the physics of entanglement
From the mathematics of entanglement.

Dual SAW Petri Spekkens



Ethernet Link Liveness



Alternating Causality protocol

Imagine an Ethernet cable connecting two SmartNICs

Liveness provides symmetric question & answer dialog:

Are you there? $\Leftrightarrow$ **Yes I am!** (First slice of frame)

Each frame may be modified *only by the receiver* when it has the frame in its possession. There is only one serializability focus per frame, but there are two frames: one owned by Alice, one owned by Bob.

- In Alice's frame we say that Alice's token is owned, but Bob's token is borrowed.
- In Bob's frame we say that Bob's token is owned, but Alice's token is borrowed.

(Following Rust rules)

MultiWay View of Link

Example of a dynamic yet stable causal structure (multicomputation reducibility) using a dual token Petri net, spanning a single bipartite link. [Quantum Ethernet. Dual SAW-Petri-Spekkens Protocol.](#)

Imagine an Ethernet cable, with each end connected to a SmartNIC -- each with a server attached to the other side of it's PCIe or CXL bus. Conventional heartbeats enable *liveness* on the link with a symmetric question and answer dialog: Are you there? <-> Yes I am!

The link continuously circulates fixed frames of shared information. The figure to the right shows two independently circulating shared frames green & red

There is thus one circulating (direction) of causality.

Each causal frame has an identity (Alice or Bob) designated in the protocol after an appropriate symmetry breaking. In Alice's frame we say that Alice's token is *owned*, but Bob's token is *borrowed*. In Bob's frame we say that Bob's token is owned, but Alice's token is borrowed.

Alice/Bob Independently initialized with a Petri Token

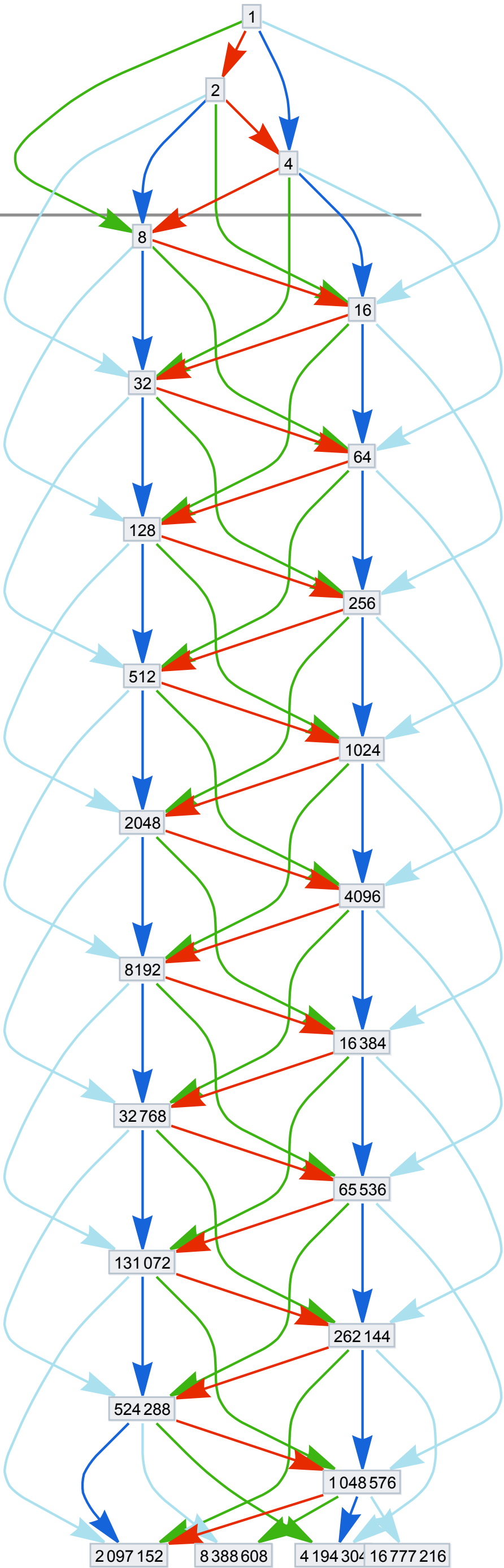
[1] Begin with Alice Alice sends two pieces of information: Red to the other side of the link, Blue to self (to merge with future operations).

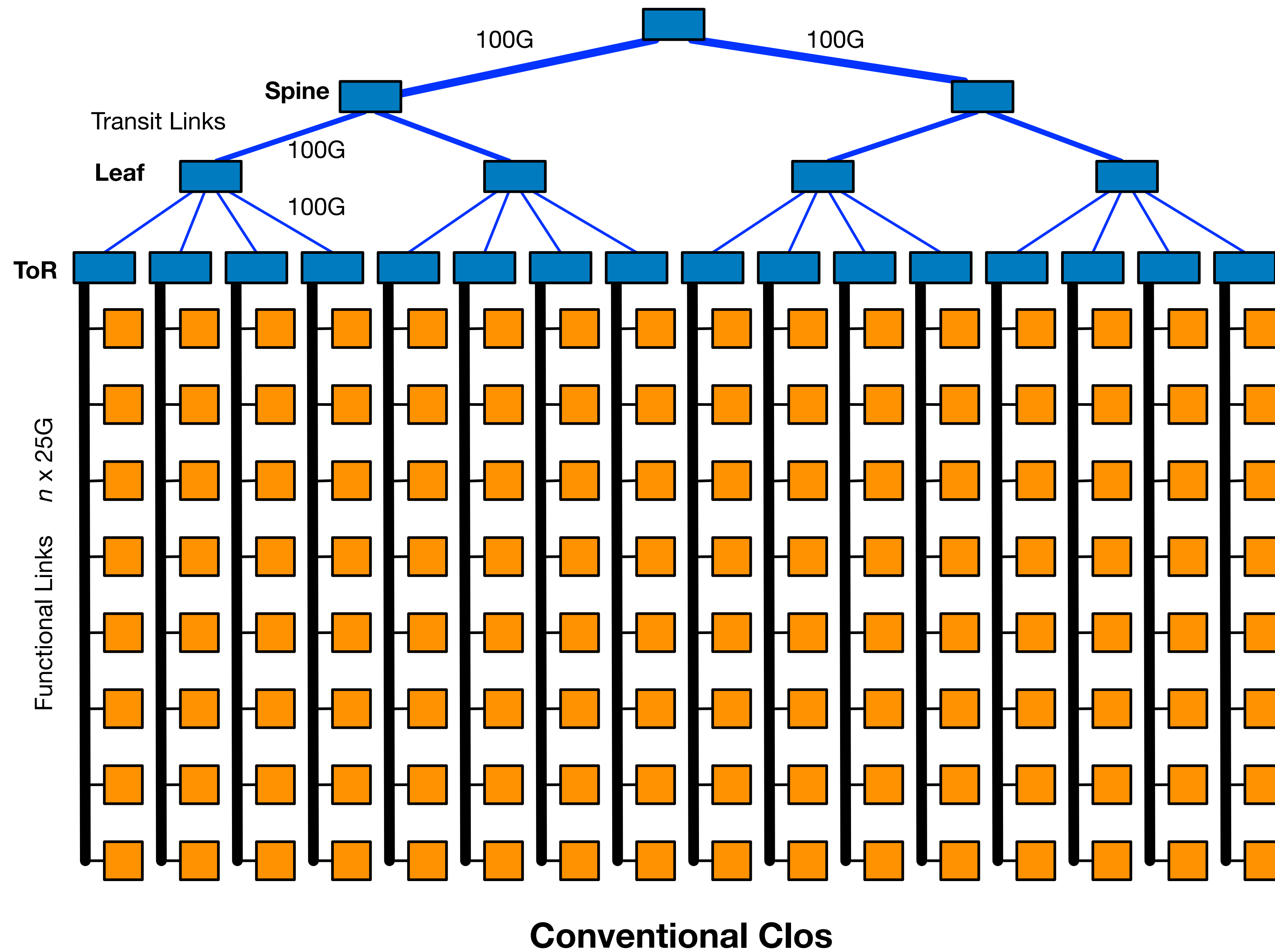
[2] Begin with Bob Bob sends two pieces of information: Red to the other side of the link, Blue to self (to merge with future operations).

[4] Alice and Bob agree on roles} (Symmetry is broken) There are now two frames in the link; one owned by Alice, one owned by Bob.

Continue with Alice} The protocol continues forever, until it is changed by the layer above. Modulo 16 counting. (starting from zero 0).

[Green] arrows Alice or Bob sending *Alice's frame* to the other side.

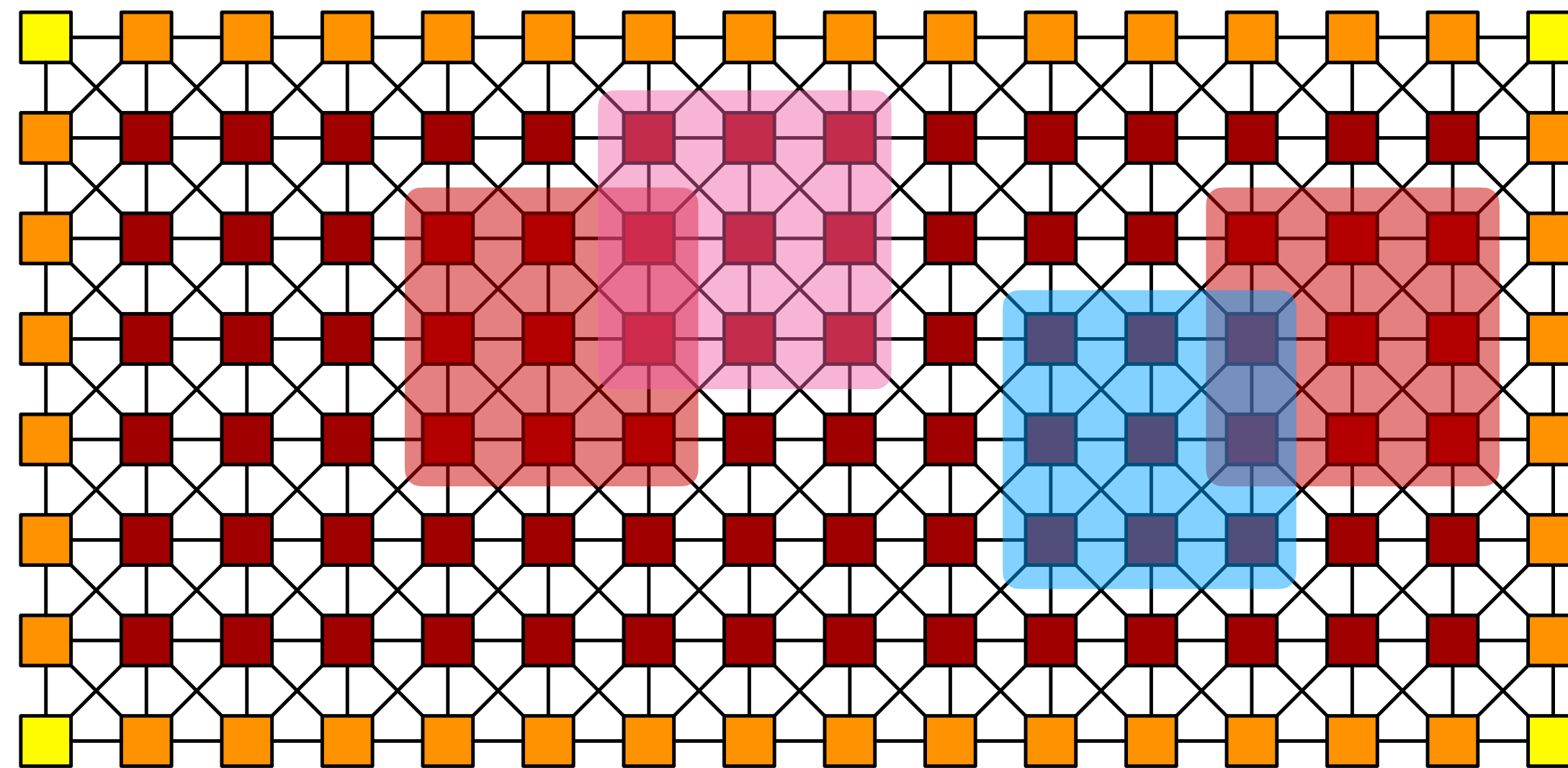




Graph (Network) Automata Foundations

8-port (Moore) neighborhood exploits physical link topology substructure within racks. But with *fragile* cabling (where links and nodes must be healed around dynamically with Local Observer View (LOV) information only)

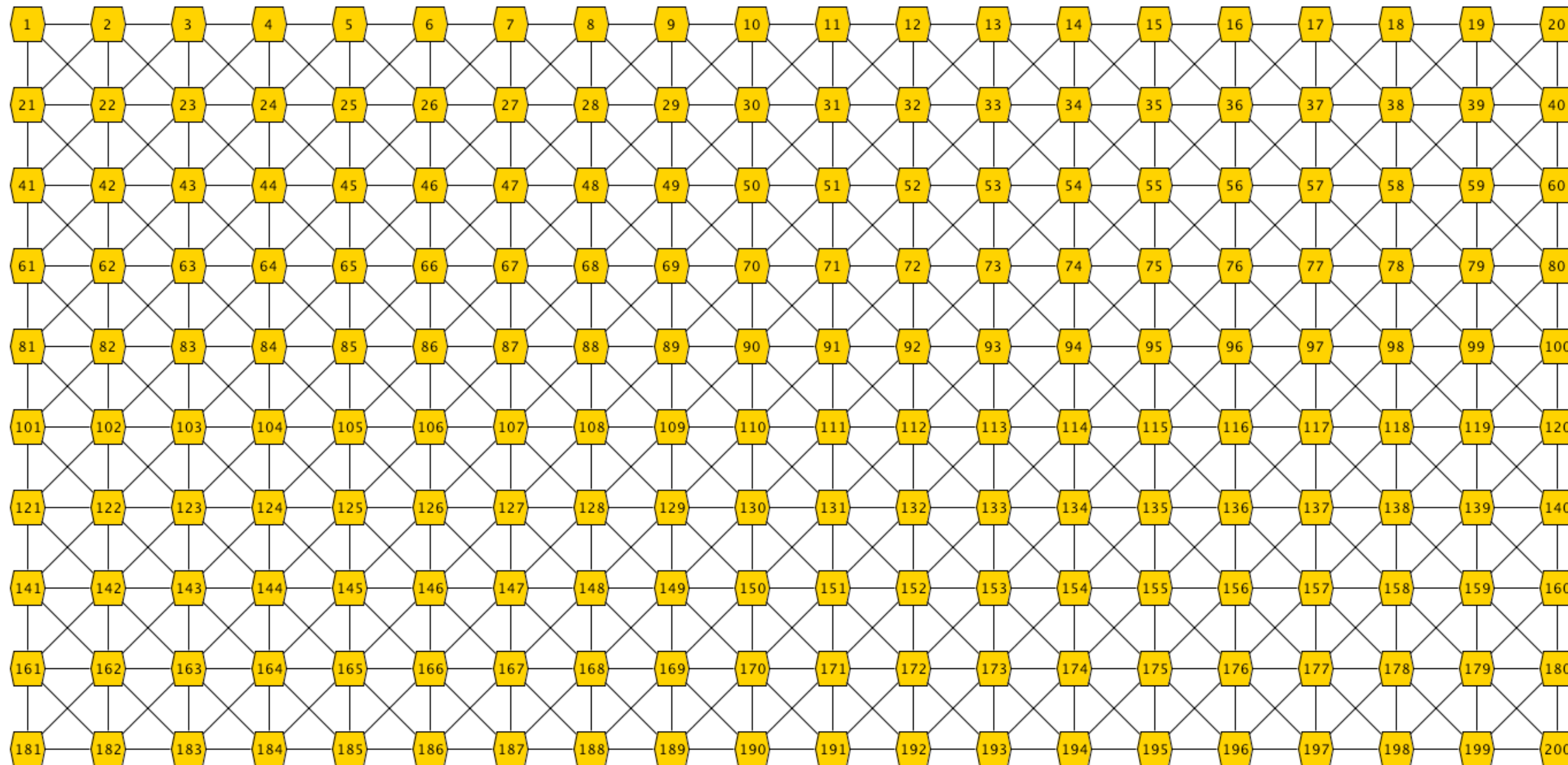
- Paves the way for highly dynamic computational strata in layers above
- Leads to a 9-Cell deployable unit of distributed computation we call a hypercell (Tile)*



**This physical structuring must not be conflated with a regular lattice on a cartesian coordinate system because our graph algorithms are highly dynamic, adapting to both the addition of new resources, or the deletion (i.e. failure or removal) of old ones. A fixed cartesian coordinate system (hypercube-like or otherwise) would be far too fragile as all labels throughout the graph would have to be replaced on failures and reconfigurations*

Regular or Irregular Lattice?

Real Infrastructures are messy

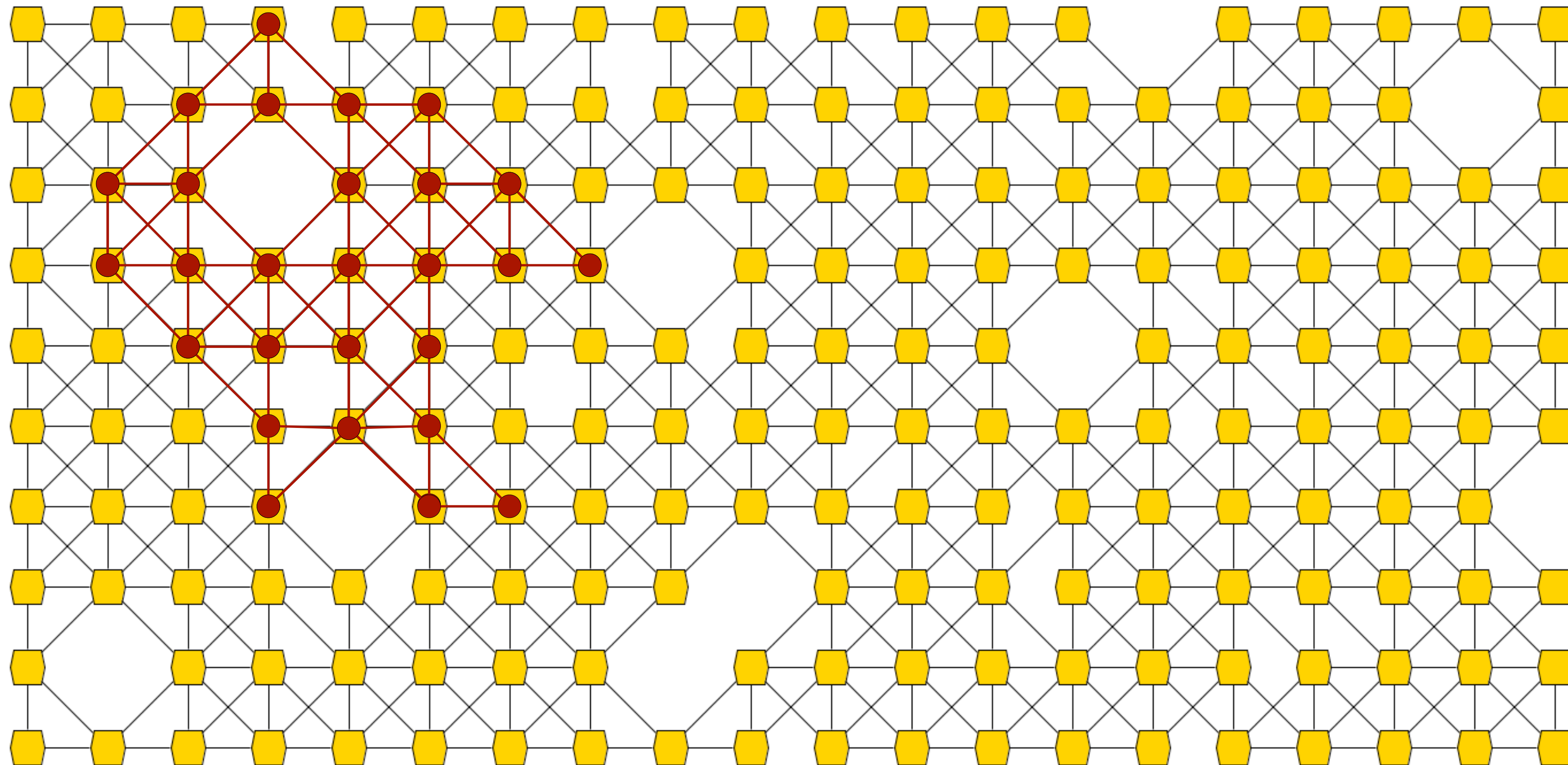


~~Absolute /
Cartesian
Coordinate
Addressing~~

Neighbor to Neighbor (N2N) Lattice

Manage on a Tree

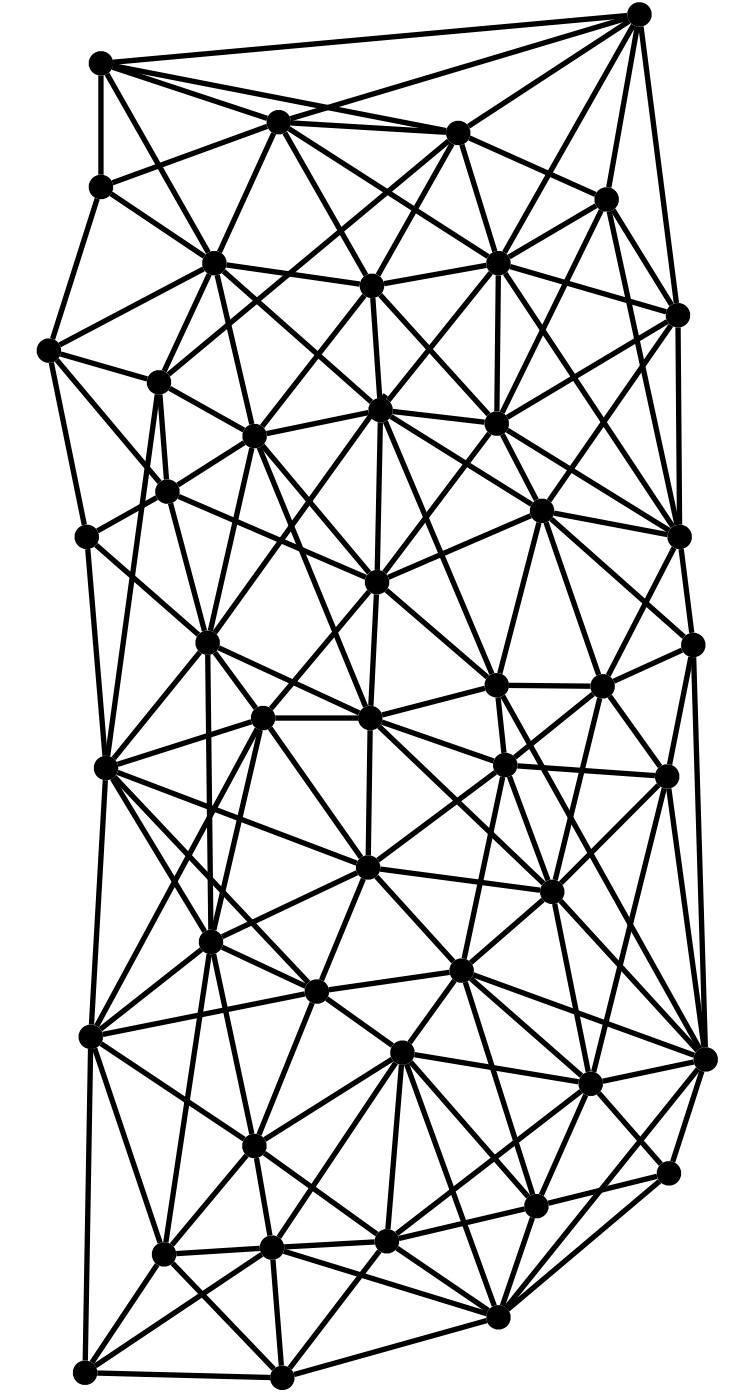
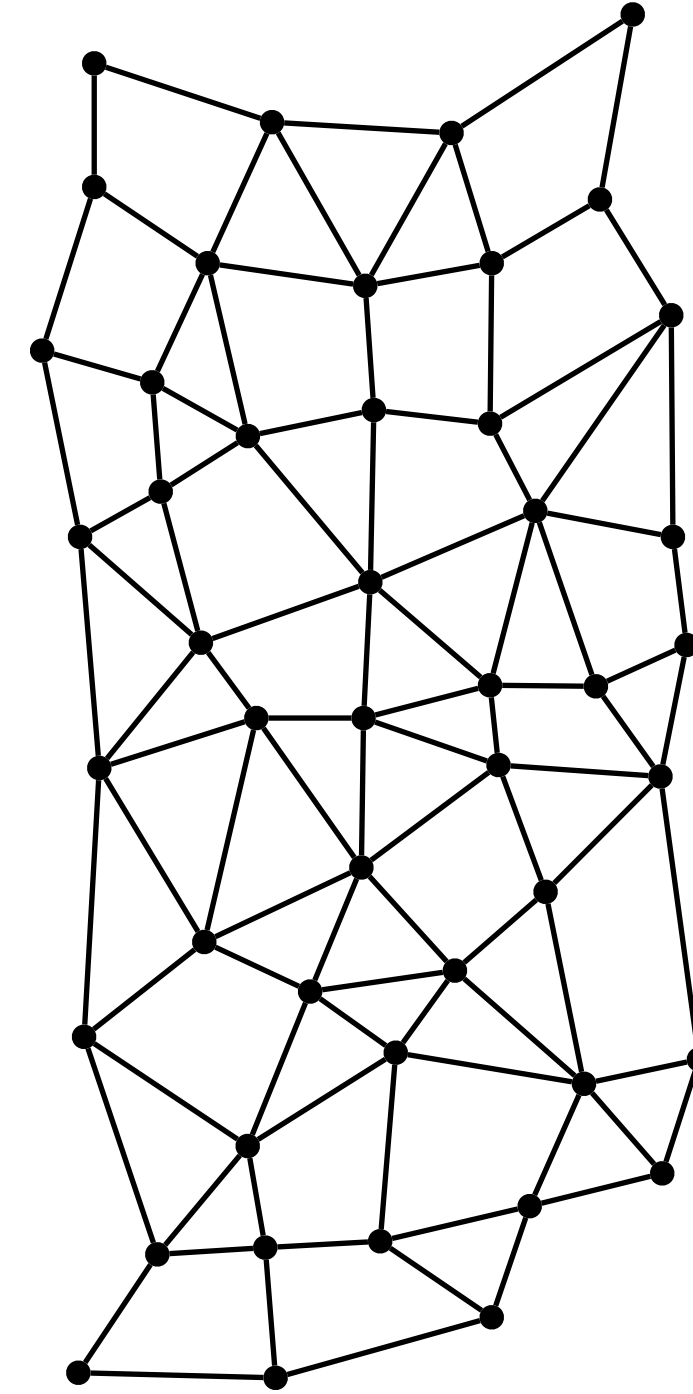
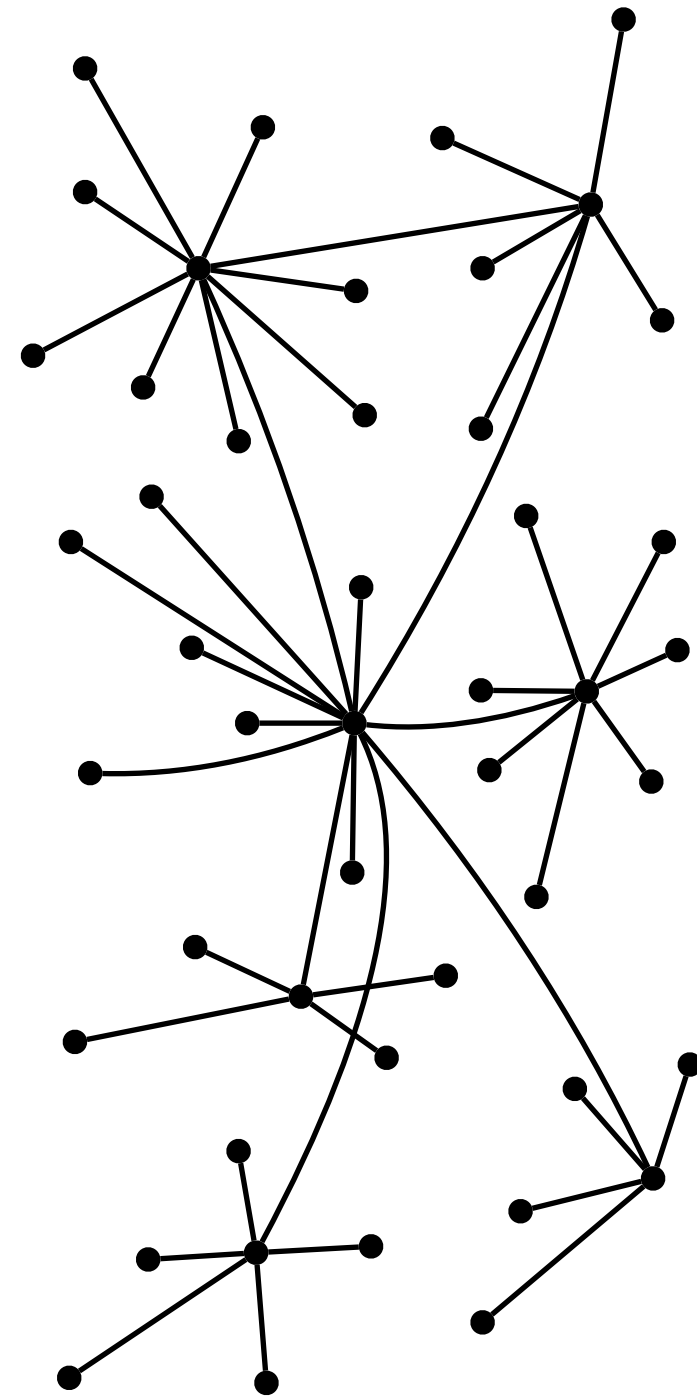
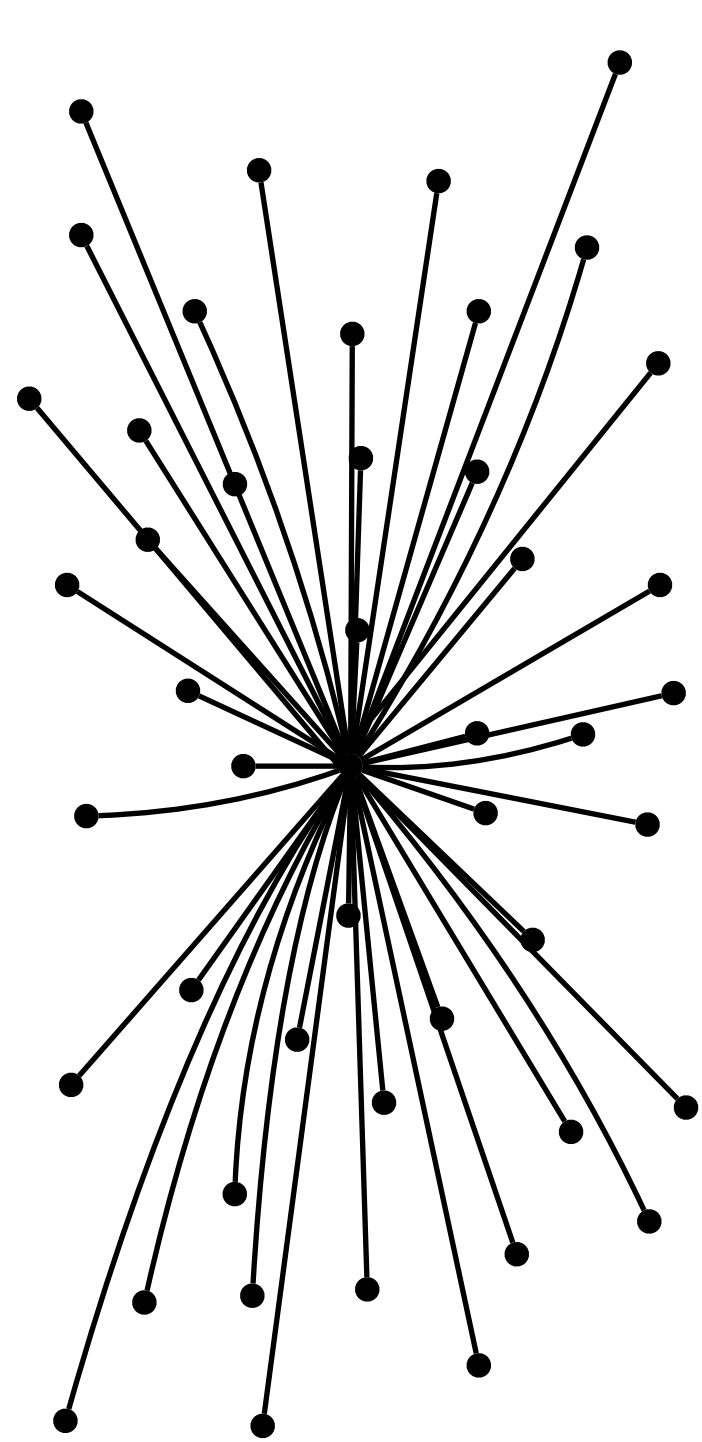
Compute on a Graph



*Relative
Address-Free
Routing*

Graph Virtual Machine (GVM)

Build and tear down 'named' graph relationships (sets/graph covers) based on properties of a cell or hypercell. Demonstrate using Paul Baran examples*



Count connectivity using: Matrix Tree Theorem & Graph Laplacian

Centralized

(47 Nodes, 46 Links)

Decentralized

(47 Nodes, 47 Links)

Distributed

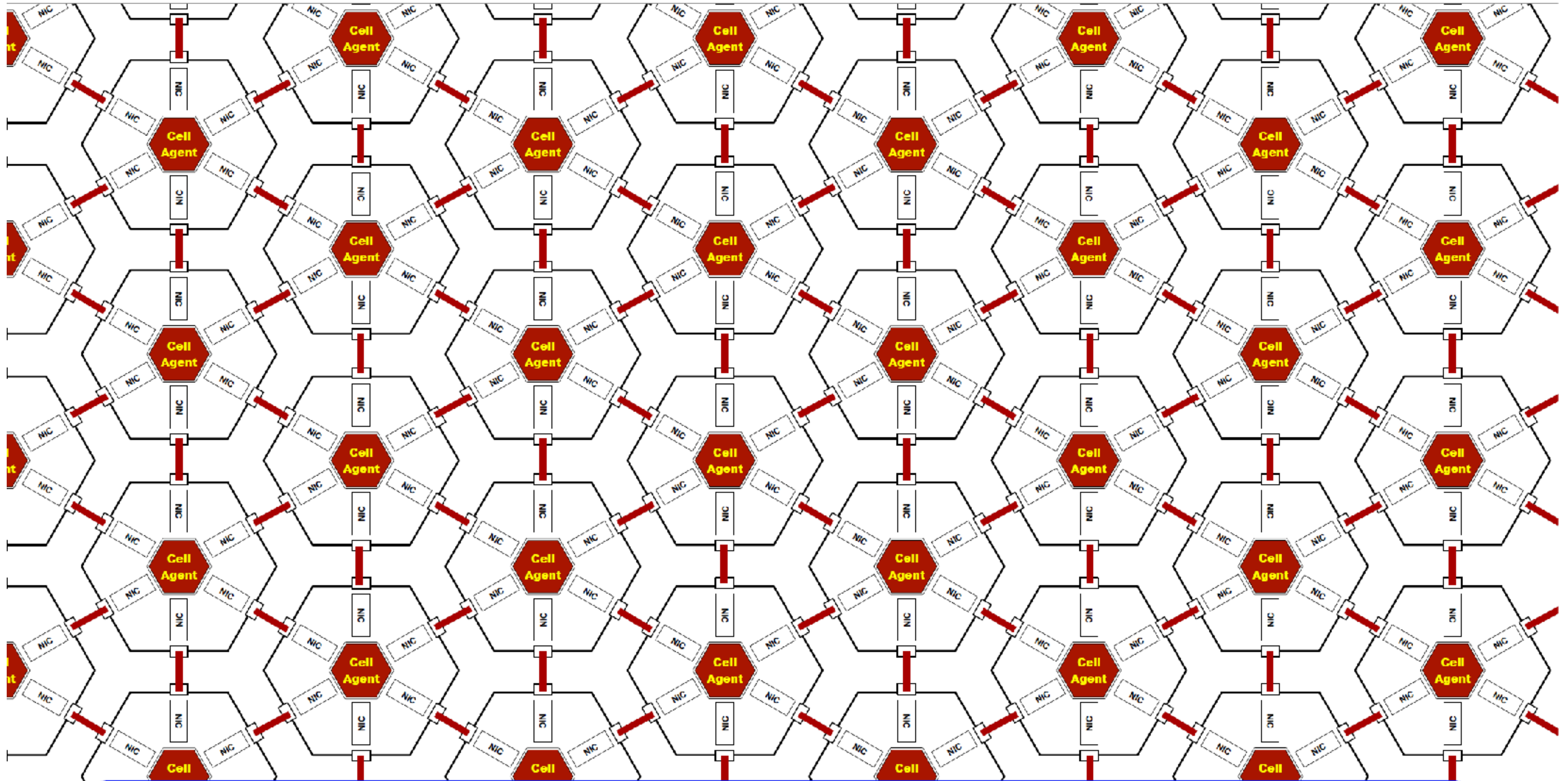
(47 Nodes, 98 Links)

Transaction Fabrix

(47 Nodes, 220 Links)

Logical Plane (mesh) of servers

Not a causal network yet (those are stacked on top)

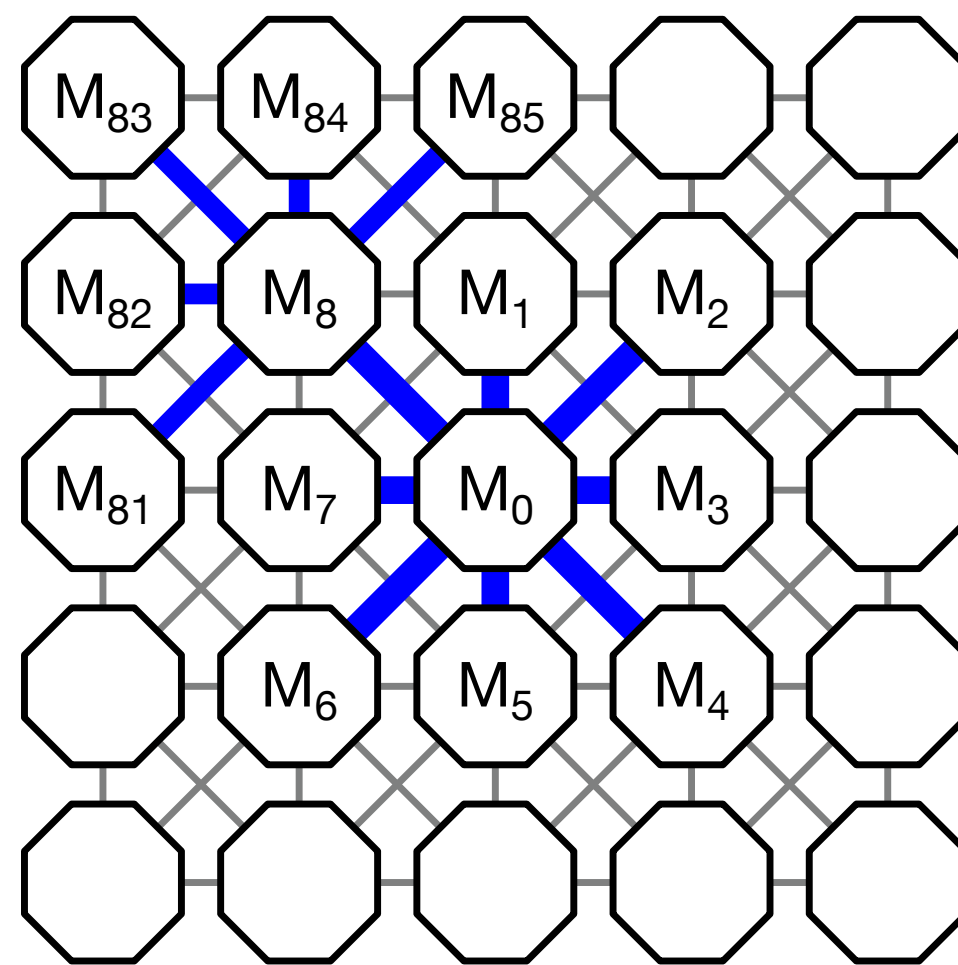


Composable (Stacked) TRAPHs

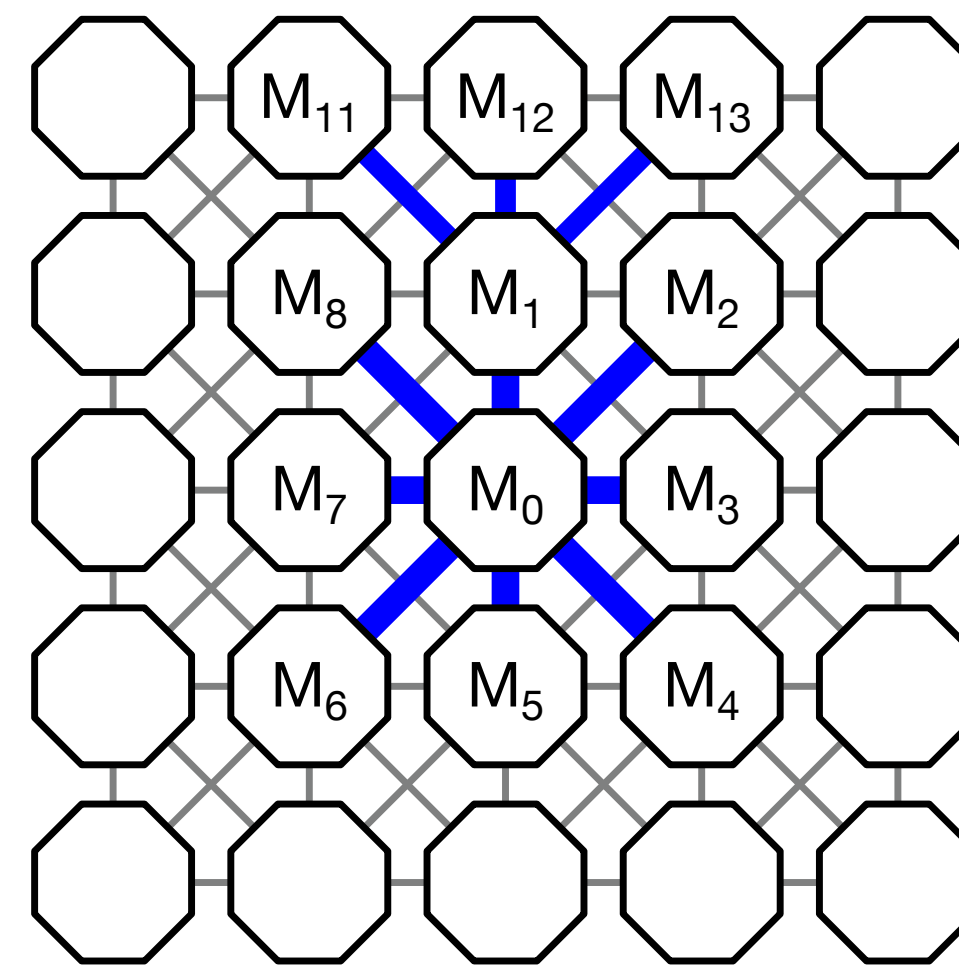
- Composable (Stacked) TRAPHs (TRee-grAPHS) Select one, or a small number of strata for experiment and visualization
- Use Cluster Liveness model to demonstrate spatial confinement
- Visualize TRAPHs for stacked trees, show physical, logical, virtual
- Demonstrate (local) distributed consensus within the tile. This 9-cell Tile also gives us a natural unit of fault-tolerance. Each cell is (by definition) the center of it's own tile
- The self-cell is thus pre-allocated in various algorithms as the center (Captain) of its own world for the purpose of initiating distributed algorithms, such as routing, consensus, autoscaling and load balancing

Hypercell Routing protocol

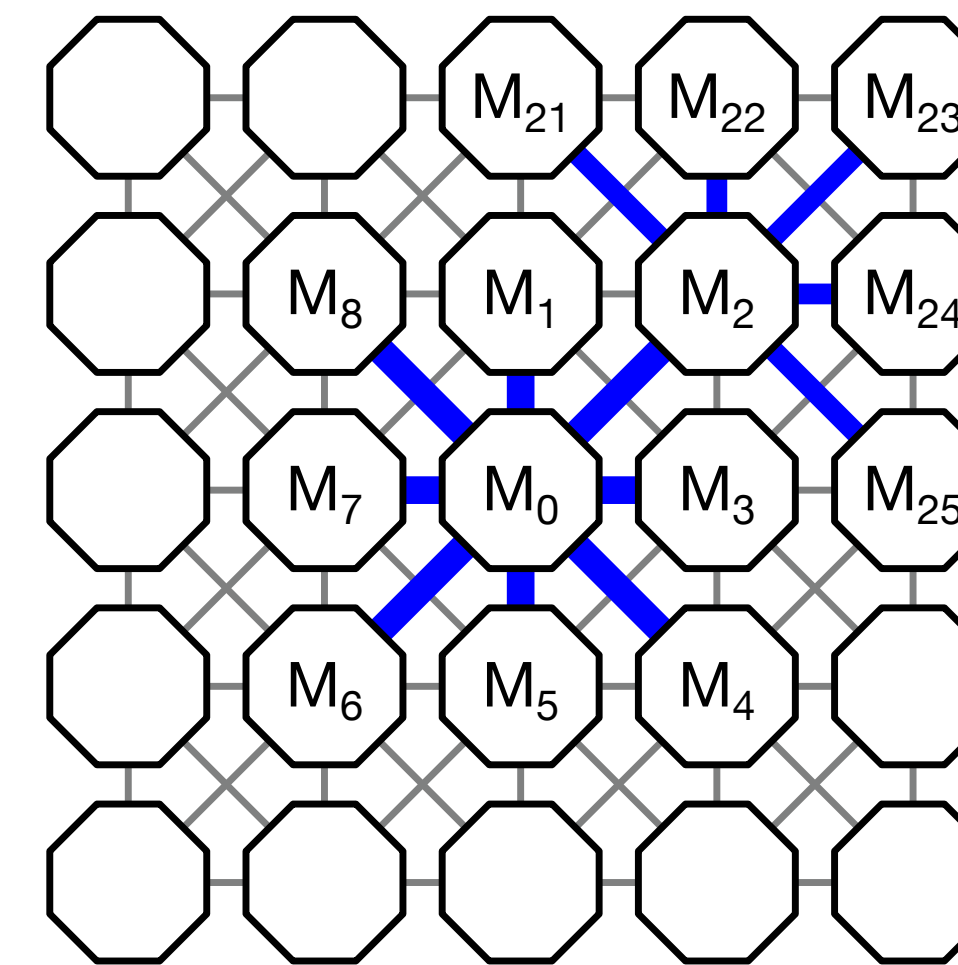
- Demonstrate Multiway routing protocol between tiles on DAGs
- Demonstrate healing around individual link failures
- Demonstrate remote consensus between tiles distributed throughout the graph, in the presence of injected failures



Tree Addressing from M8



Tree Addressing from M1

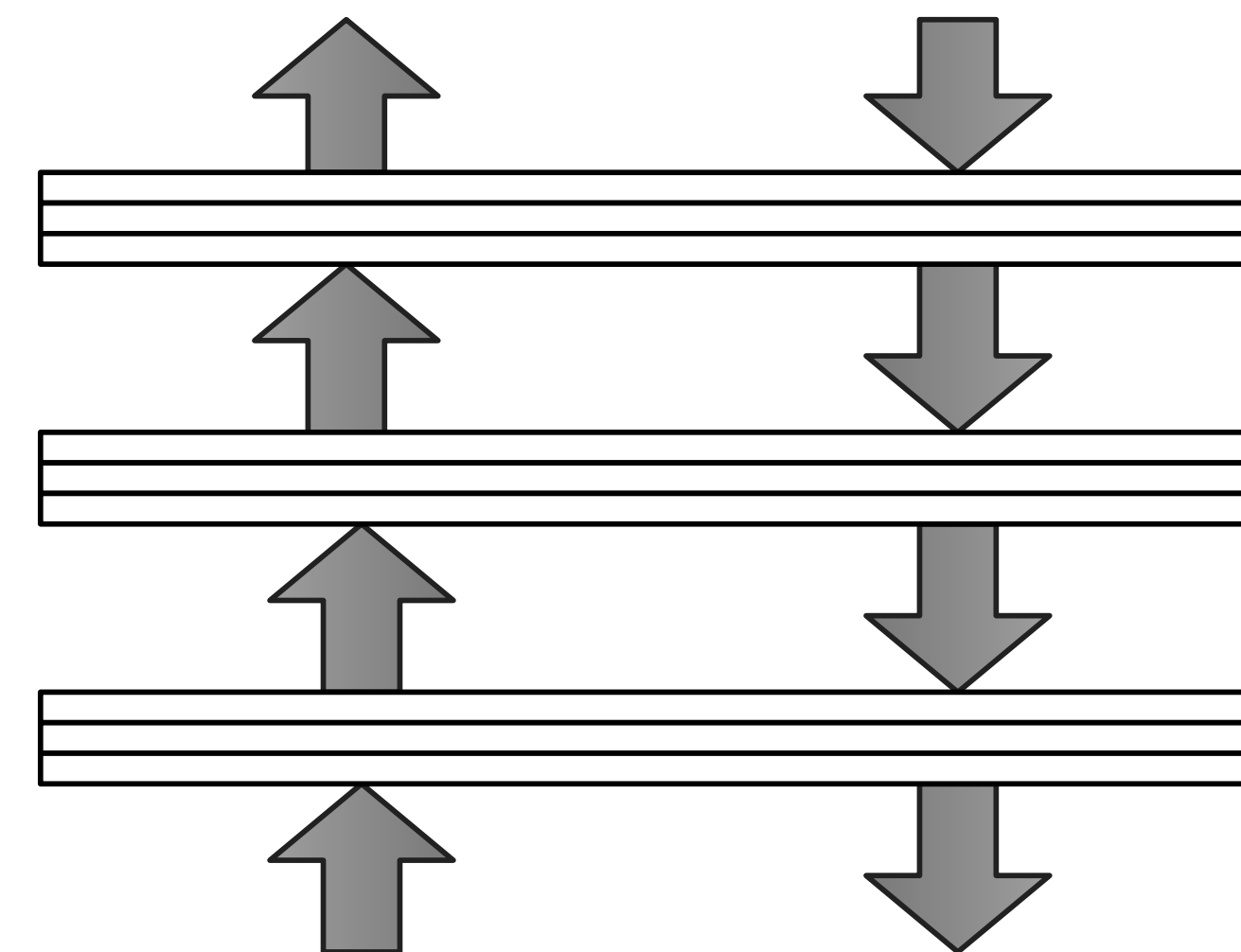
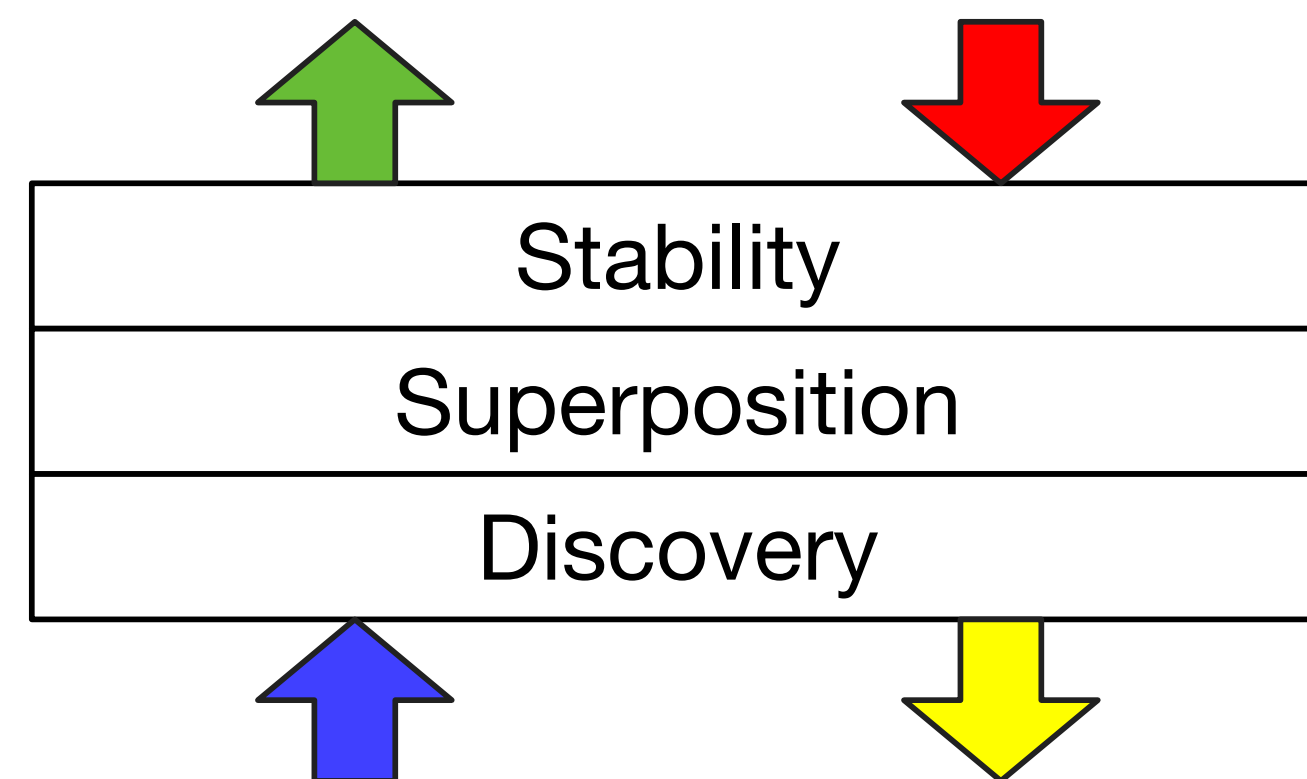


Tree Addressing from M2

Tree Addressing is *Relative* — It looks different from each Tree's perspective

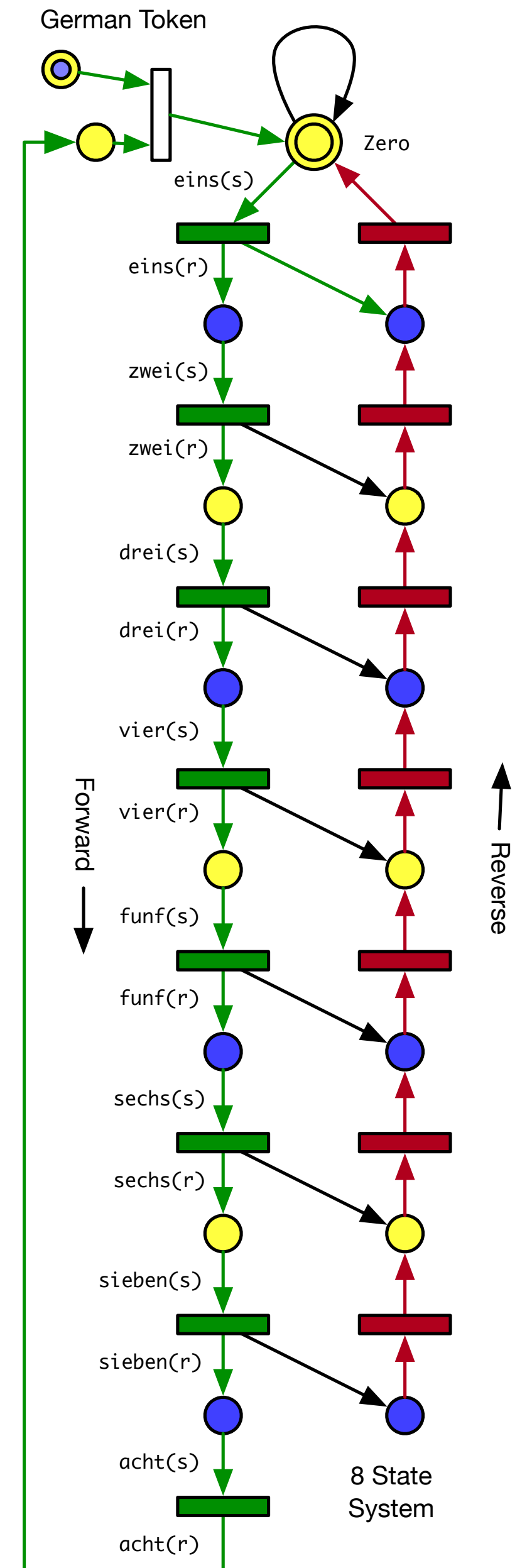
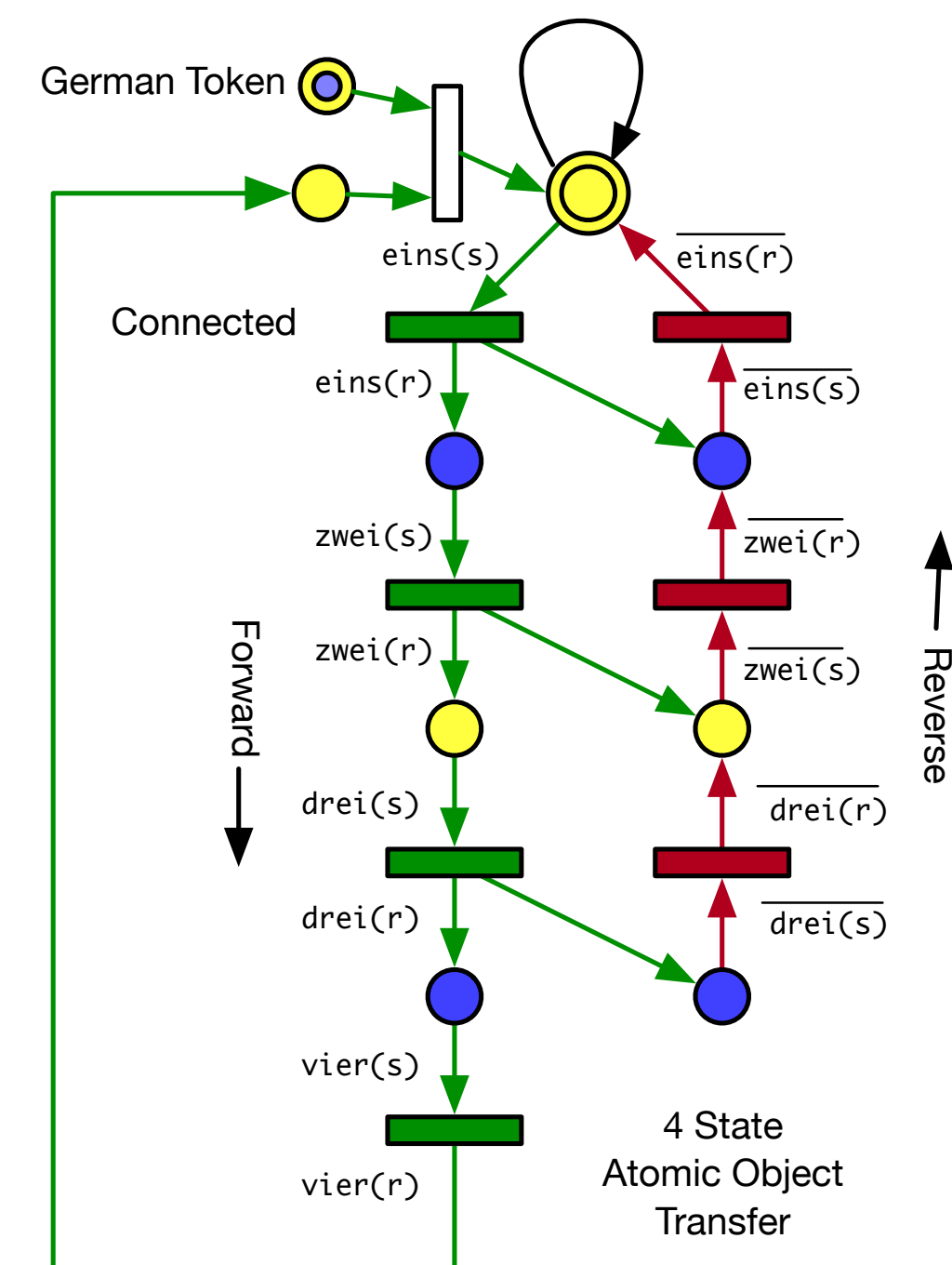
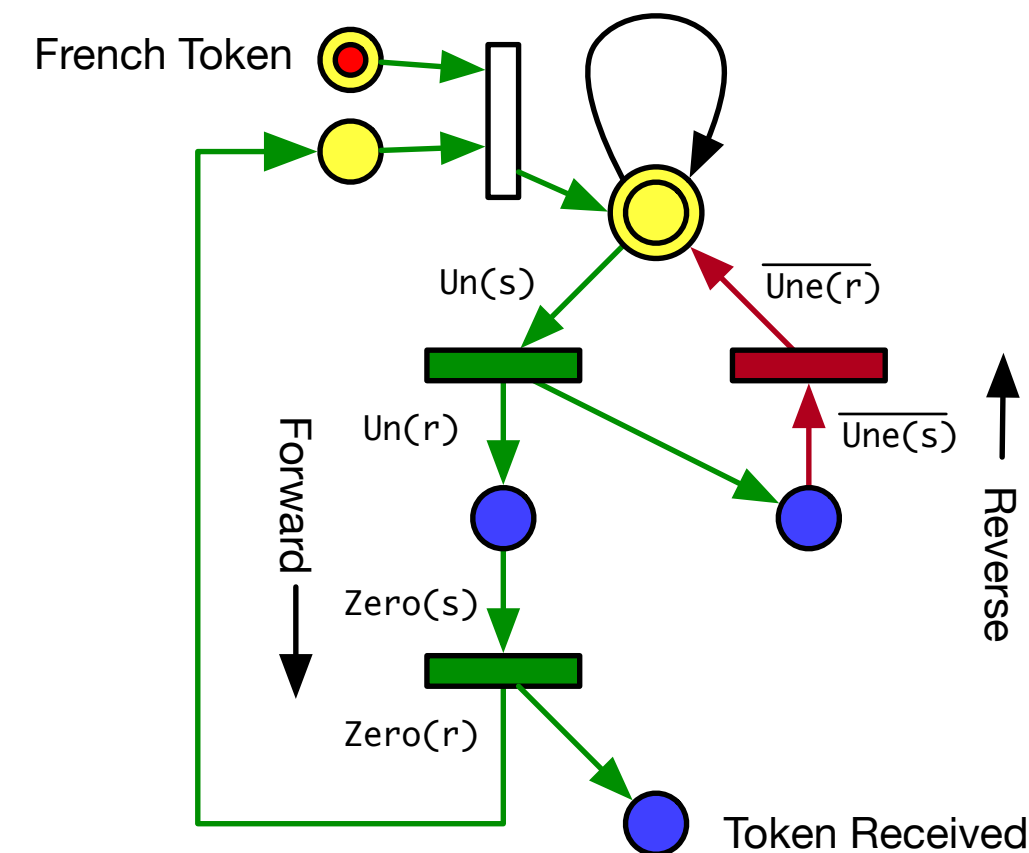
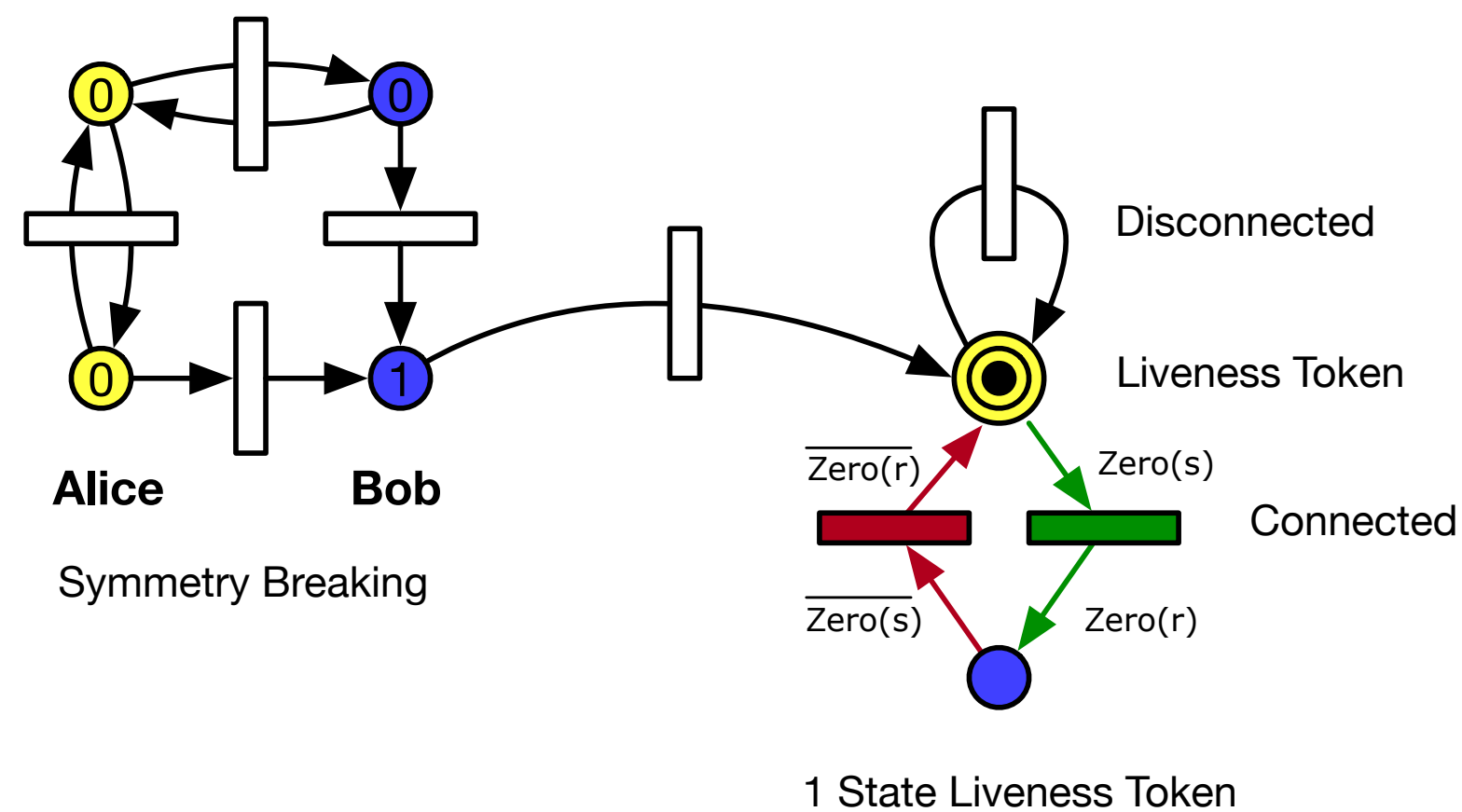
Graph Stratification

- Demonstrate coarse graining where hypergraphs represent multiple physical nodes 3×3 tiles which represents cooperative computational and failover entities
- 'Zoom out' to whole datacenter (all nodes), or 'Zoom in' to smaller sets of nodes in hypergraph style consolidation based on the same set of physical vertices and physical edges. Hypergraph can also 'go down' into the physical nodes and connect with sets of execution entities in a potentially hierarchical structure such as virtual machines, containers and lambdas
- Demonstrate 'Kubernetes replacement' for configuration and deployment

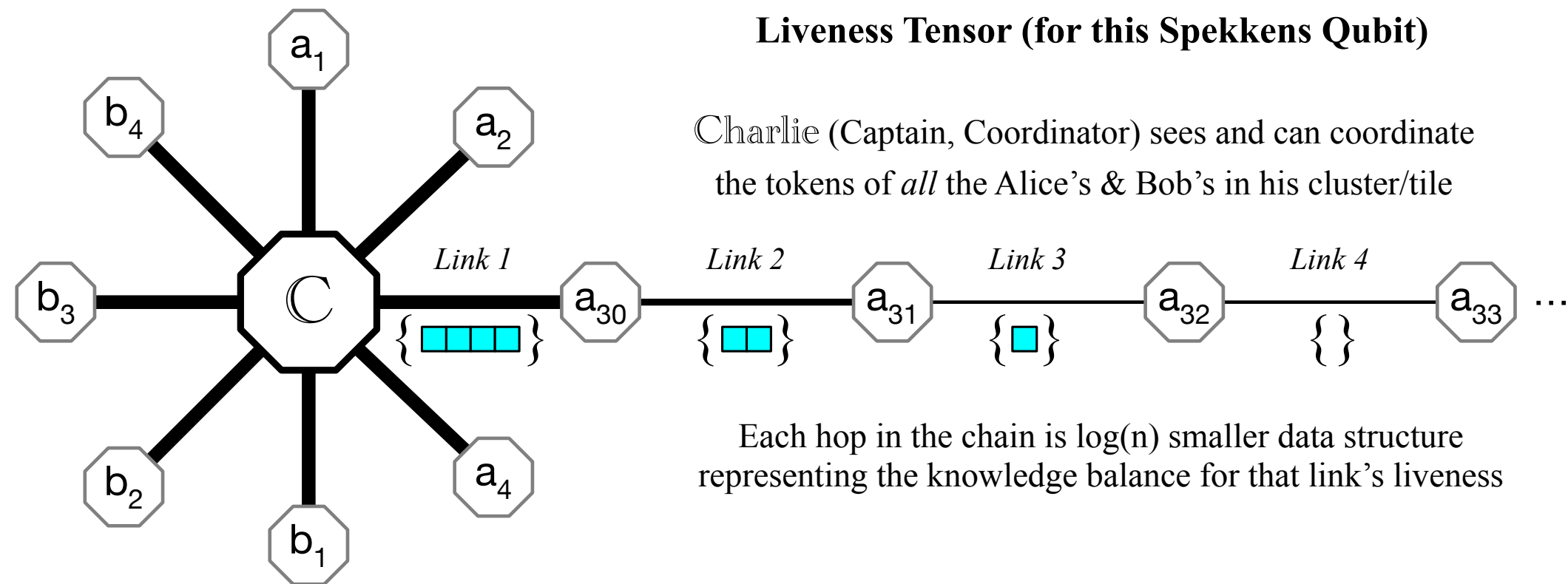


Token Dynamics

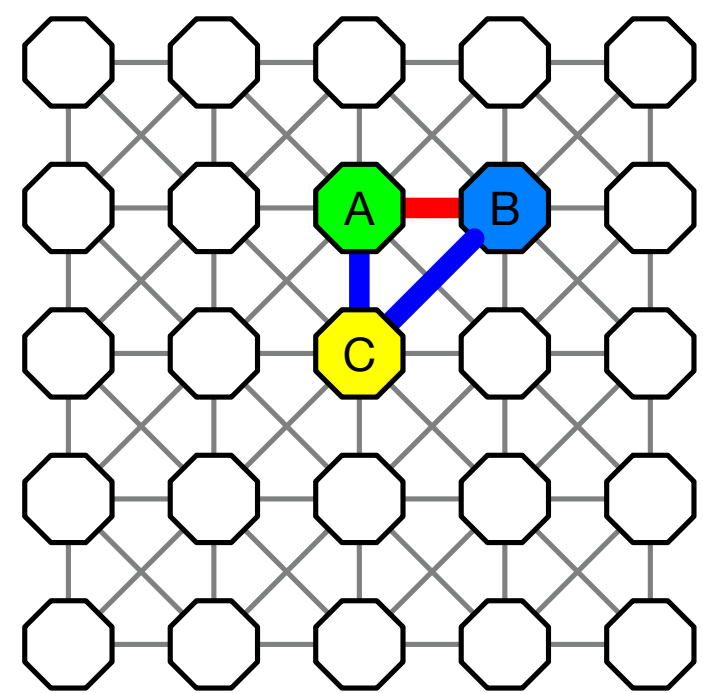
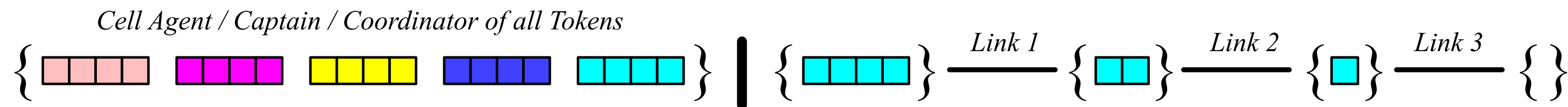
Conserved quantities/exchanged quantities/reversal recovery/lost baggage on *TRAPHs*. Include measurements such as graph resiliency, total counts for each token color. Include consistency checks to be tested after fault-injection of each kind of hazard (magnitude and phase errors in Spekkens states)



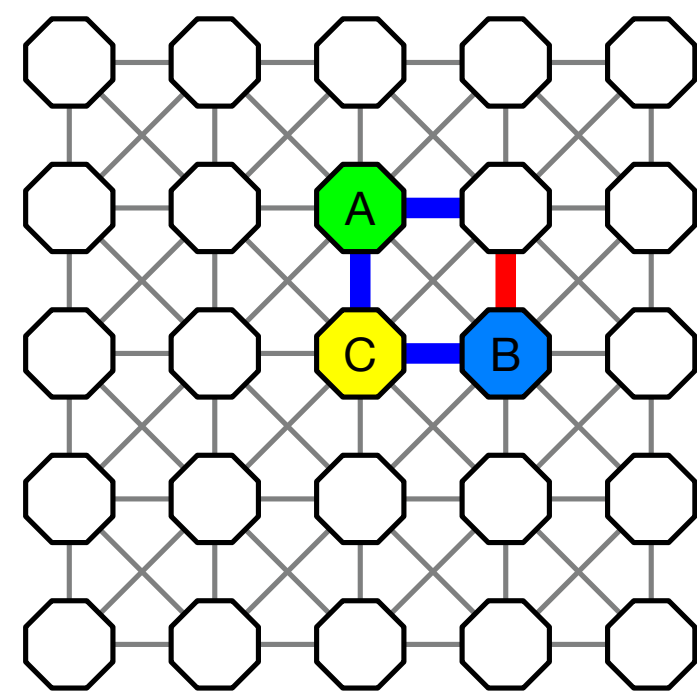
Liveness Tensor



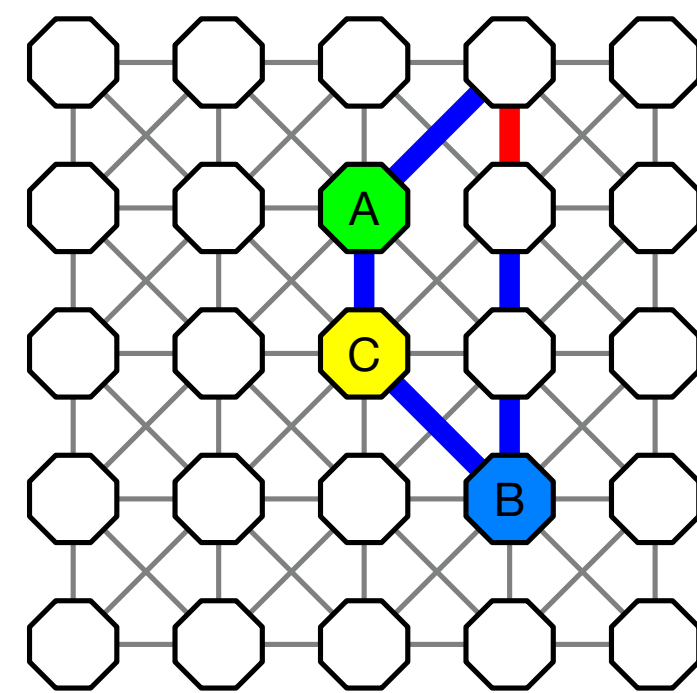
Local topology awareness 1, 2 hops out, as basis for GVM, routing (and healing), and consensus engine. Demonstrate Raft and Paxos consensus protocols using single and multi- token Petri net. Demonstrate Reversible Constructor for 2PC, 3PC & 4PC



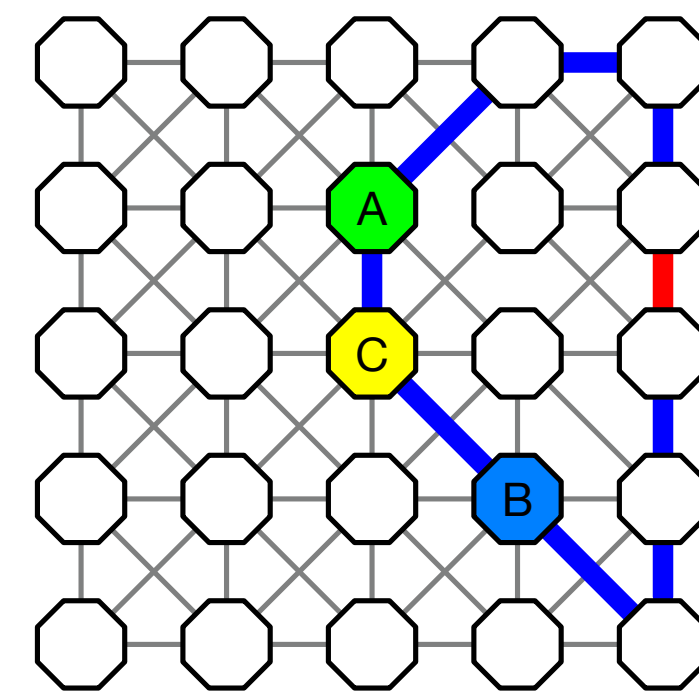
(a) 1st Hop Failure and Bypass



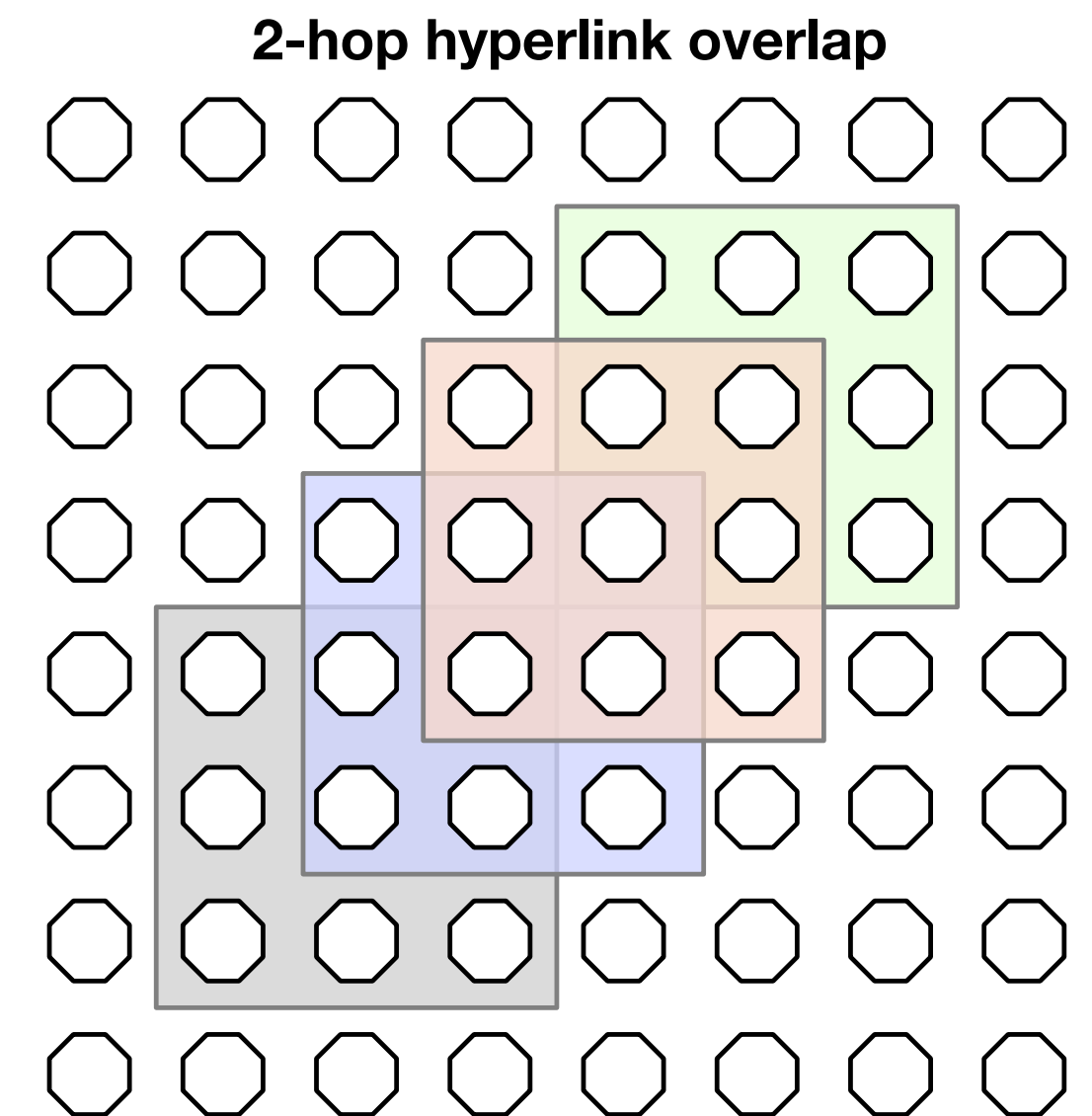
(b) 2nd Hop Failure and Bypass



(c) 3rd Hop Failure and Bypass

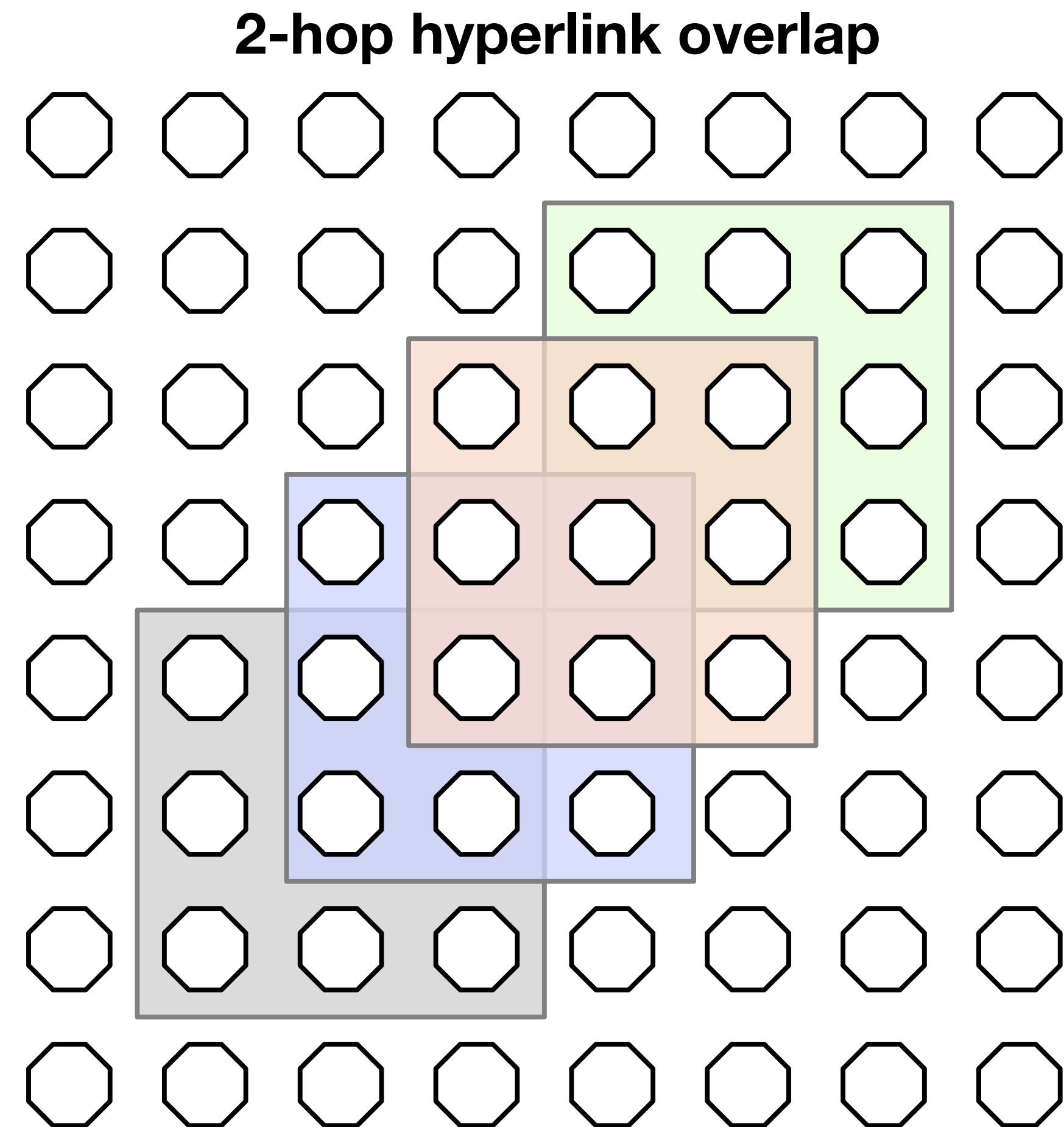


(d) Hop Failure and Bypass

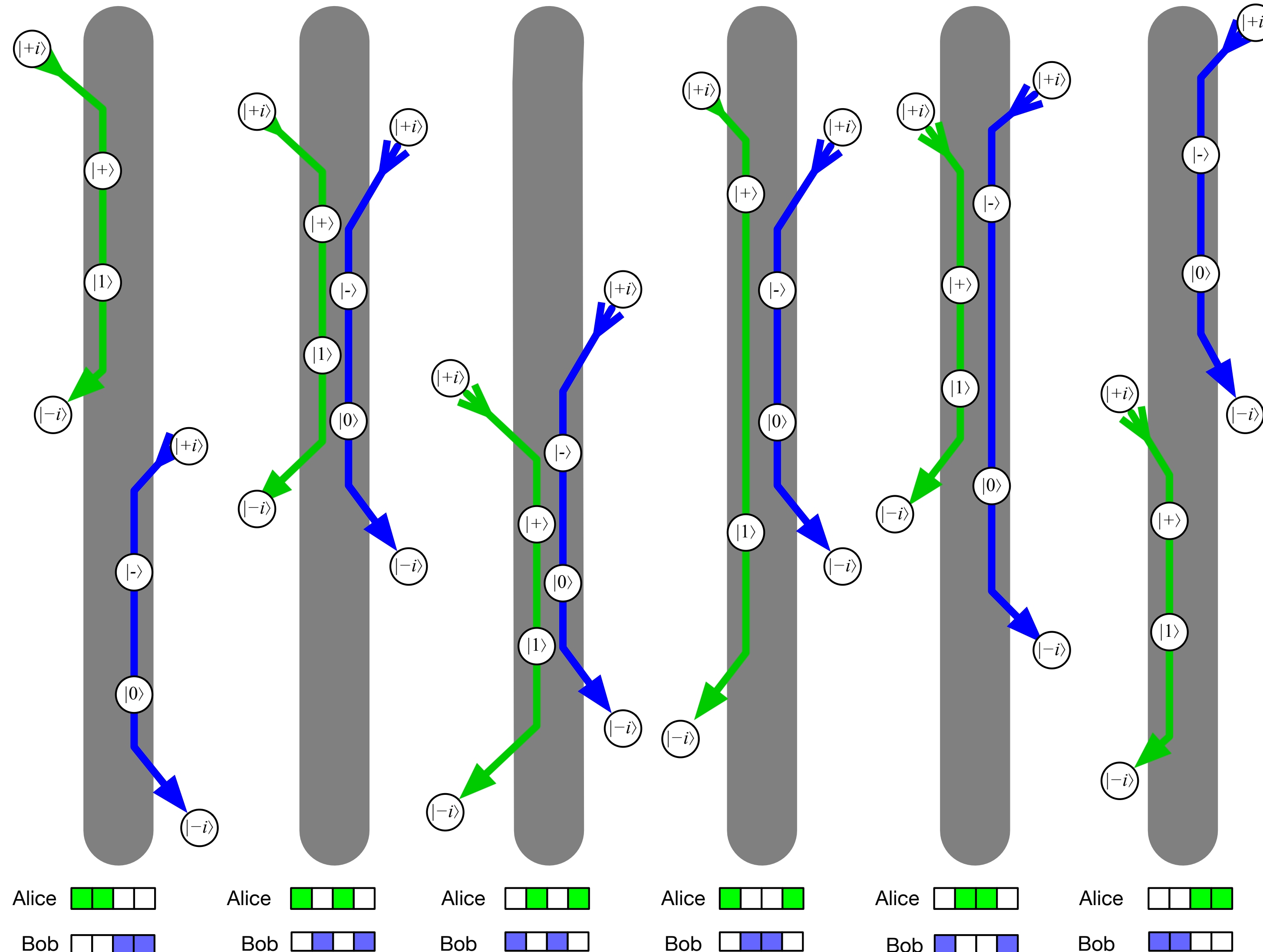


Liveness Tensor

- On the Link
- In the Cell
- In the Hypercell (8-ports)
- On the Trees
- On the Graph
- On the Hypergraph



Serialization in the FPGA SerDes

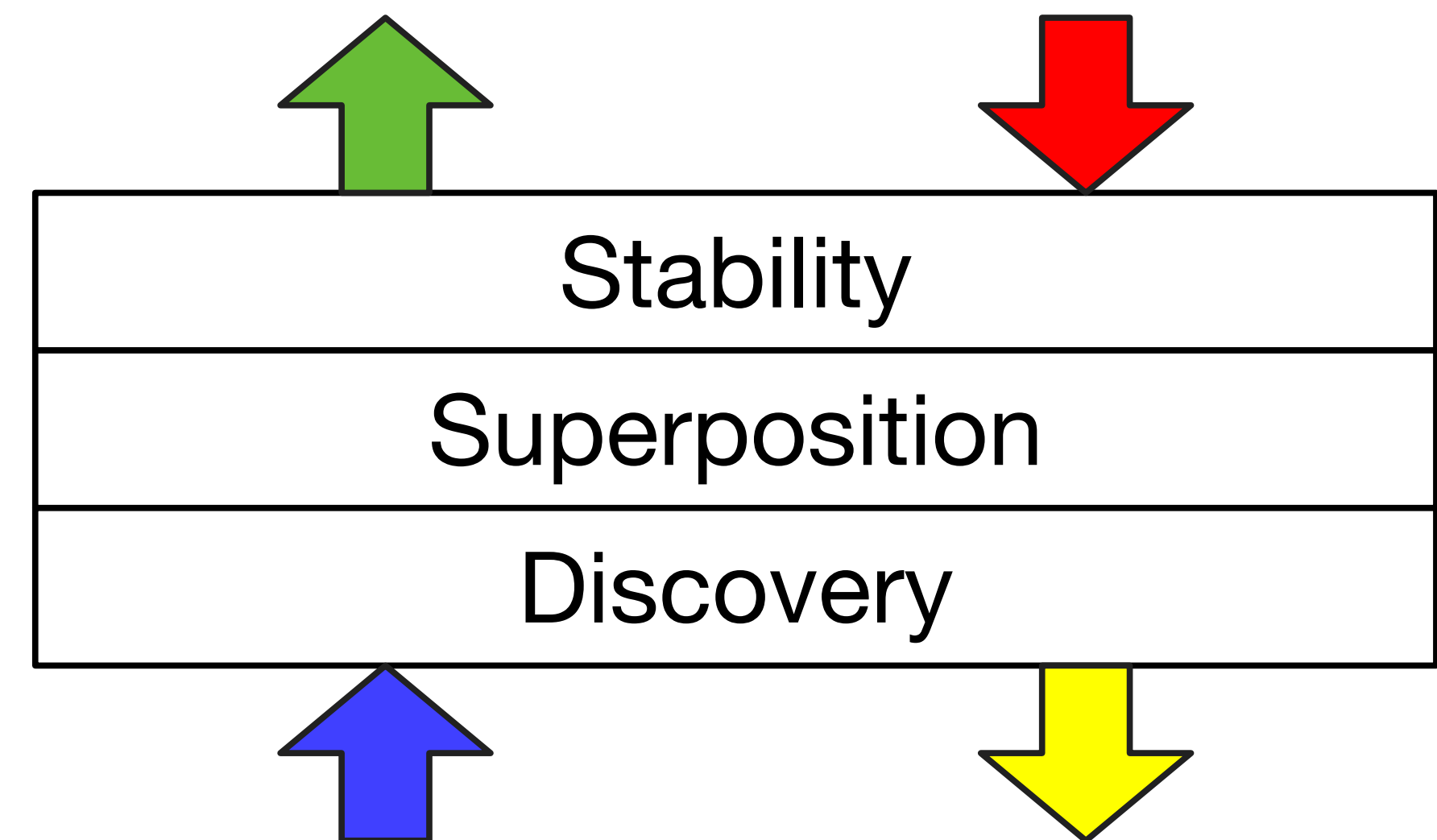


We proactively reorder events to match application constraints, eliminate race conditions, and prevent write skews before the database/application sees them as an error

Petri-Spekkens Token Orderings in the FPGA SerDes and Disjoint Pairs

Causal Dynamics

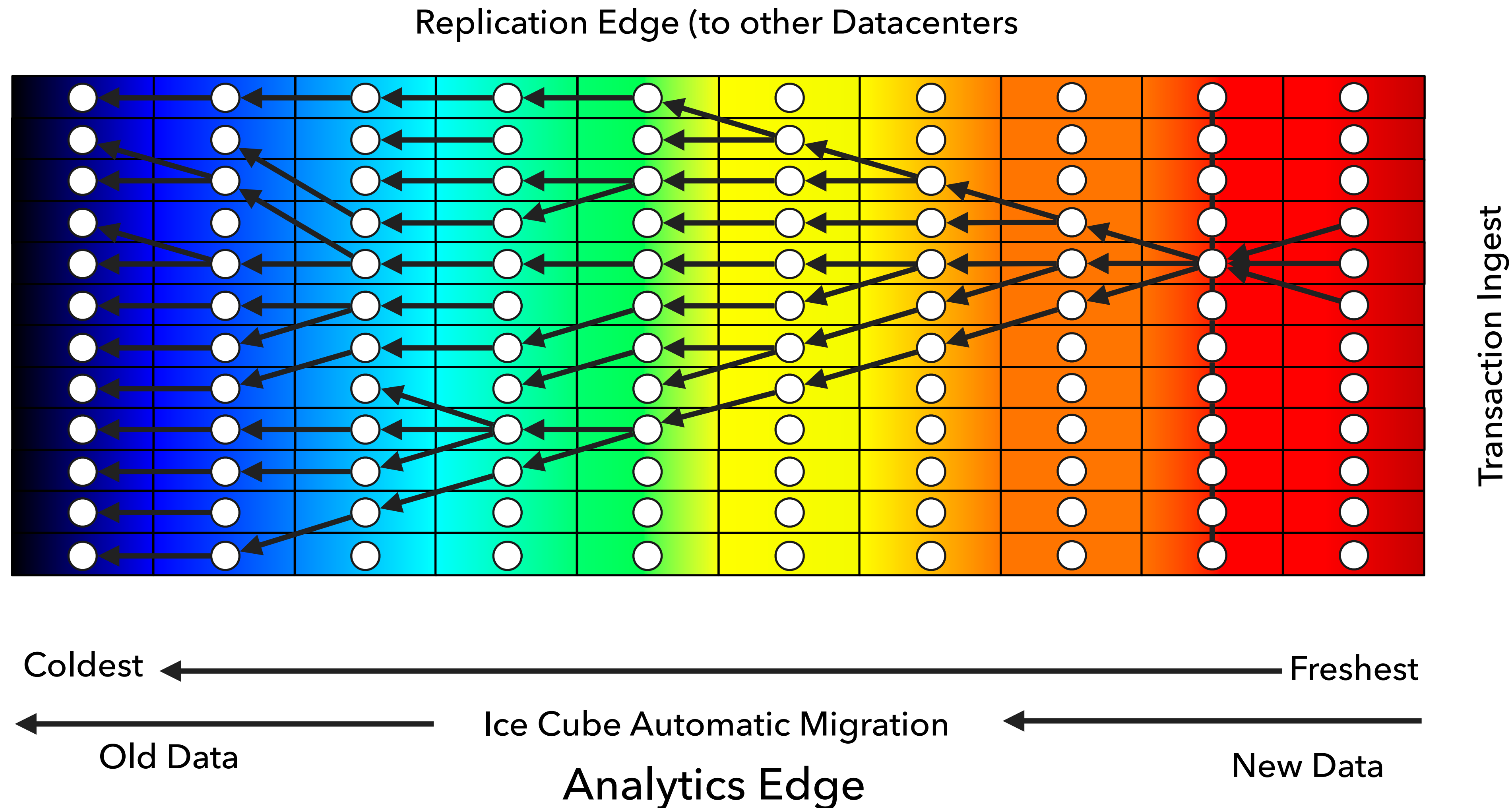
- An important goal of this design is to enable the setup, manipulation, and metastable strata of multi-level causal structures for Microservices. Jonathan Gorard, our resident expert, is cofounder of the [Wolfram Physics Project](#)
- We see an opportunity to revolutionize distributed systems in datacenters *everywhere*, by developing multiway algorithms to predictably ‘re-order causality’ *invisibly* and *indivisibly* in the FPGA substructure (SmartNICs)



See Emily Adlam: [Is There Causation in Fundamental Physics? New Insights from Process Matrices and Quantum Causal Modelling](#)

OLTP Without Timestamps

Don't use Timestamps, use Causal Trees



We Make Transactions Reliable

Reliable Substructure Clusters

Reversible transfers at line rates, with ultra-low Latency

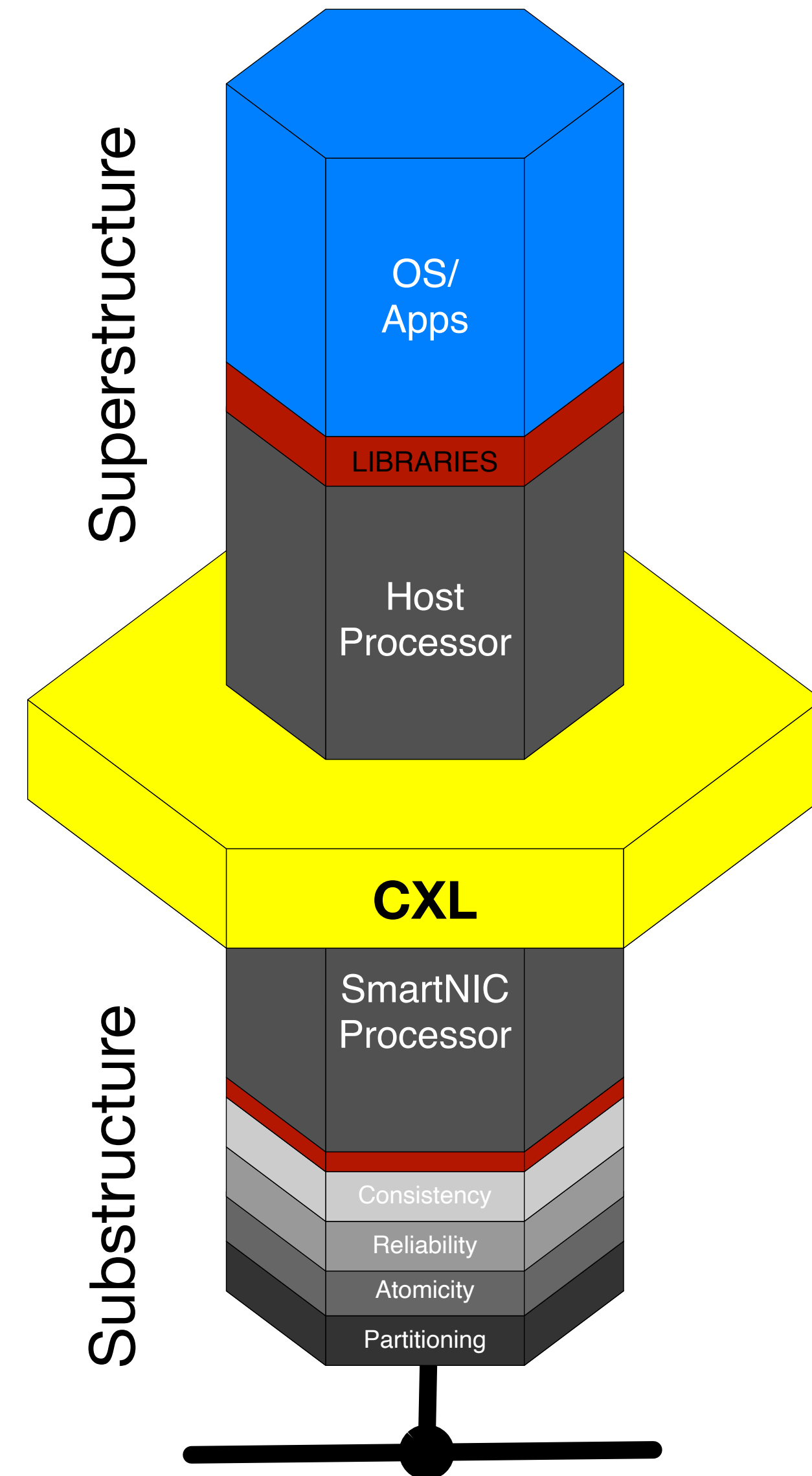
No tradeoff of Bandwidth and Latency

No Metastable Failures

FPGAs do not have a halt state!

They just run. Just circuits. Stuff goes in comes out, no halt states in between

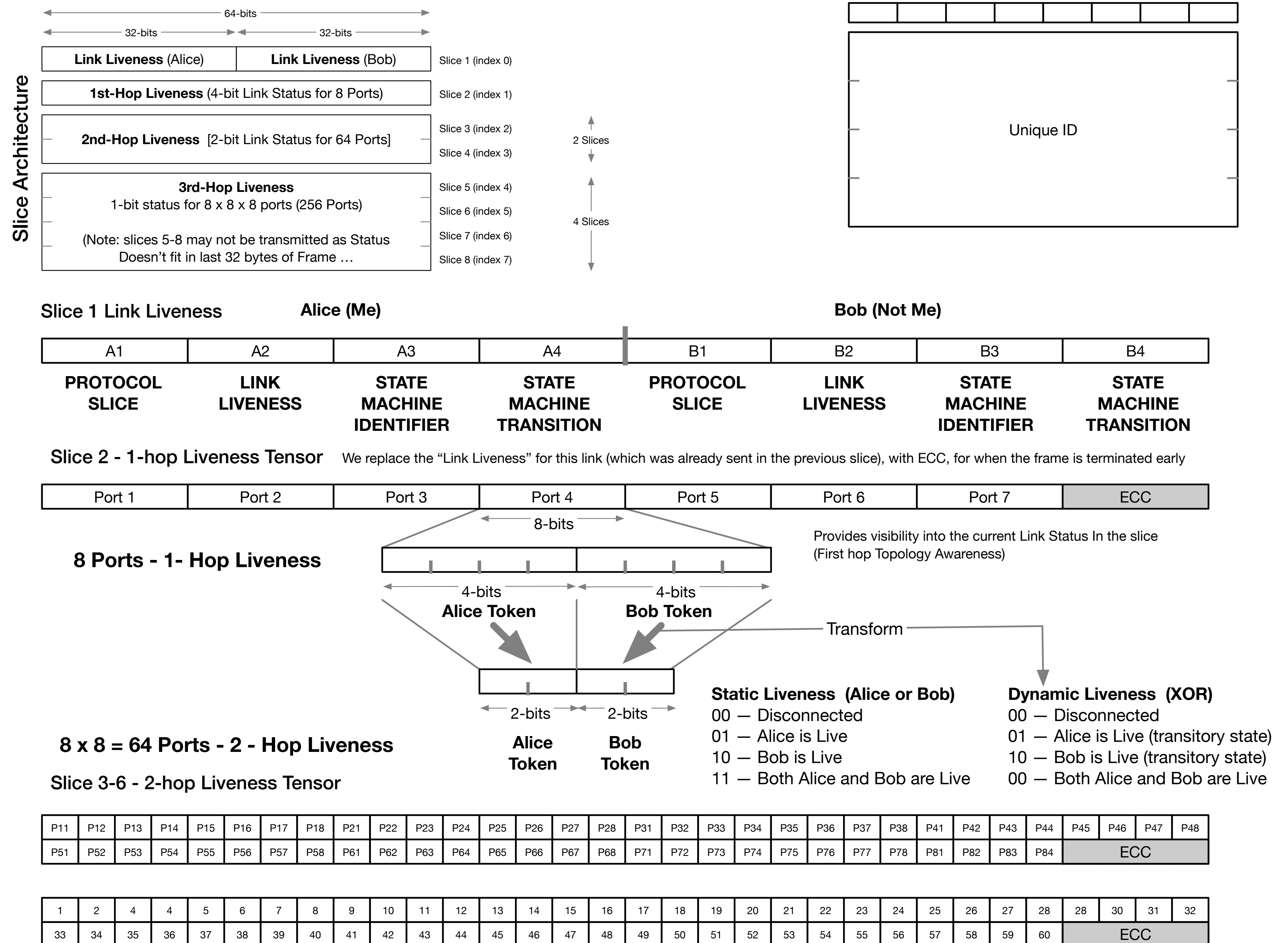
Unlike ASICs, they don't take years to create



Principles

- Timestamp-free event-only link protocol, with formal verification
- Address-free tree routing protocol, with local and remote asymmetry
- Reversible subtransactions within a reversible transaction
- Ethernet
 - Reliable Address-Free
 - Distributed Autonomous
 - Secure Address-Free

Slice Engine Architecture ECC



For David: Next:

Weekly 1/1's (with Dean)

1. This introduction
2. Goals, Timeline, Funding Profile
3. Go through Applied Graph Theory (Chiplet Summit)
4. Grid presentation (Product, Features, Market)
5. Project Definition